

## Jour 2 : Créer et optimiser un PMC sur données structurées

Hier on a tout fait à la main : le forward pass, le gradient, la backpropagation. C'était voulu. Maintenant vous savez ce qui se passe dans la boîte. Aujourd'hui on change de vitesse : Keras prend en charge la plomberie, et nous on s'occupe de ce qui compte vraiment (diagnostiquer, optimiser, régulariser). Le livrable du jour : trois classifieurs sur trois datasets réels, avec des logs TensorBoard propres et un repo qui documente vos résultats. C'est exactement ce qu'on attend d'un profil data en entreprise.



### 1 - Révisions rapides

Allez on rallume nos neurones !

- Qui peut citer les trois composantes d'un PMC ?
- On avait deux familles de fonctions d'activation hier : lesquelles, et pour quelle tâche ?
- La backpropagation calcule quoi, exactement ? En une phrase.

On a tous en tête le schéma du PMC d'hier ?

Hier on a écrit 50 lignes de numpy pour faire quelque chose qu'on va maintenant faire en 10 lignes de Keras. Est-ce qu'on a gaspillé notre temps hier ? Non : vous savez maintenant ce que Keras cache. C'est exactement la différence entre conduire une voiture et savoir ce qui se passe sous le capot.

Avant de coder, s'assurer que [TensorFlow](#) et [Keras](#) sont disponibles. [GridSearchCV](#) et [KerasClassifier](#) apparaissent plus loin; ils permettent d'encapsuler un modèle Keras dans le framework scikit-learn pour optimiser les hyperparamètres.

---

## Planning de la Journée

---

1. Révisions rapides J1
  2. **PMC avec Keras pas à pas** : de zéro à un modèle qui tourne
  3. **Diagnostic avec TensorBoard** : lire les courbes, repérer l'overfitting
  4. **Tuning des hyperparamètres** : grid search, random search, keras-tuner
  5. **Régularisation** : L1, L2, batch normalization, early stopping
  6. **À votre tour** : trois datasets structurés, 9 phases continues
  7. Récap Jour 2
- 

## 2 - PMC avec Keras/TF, de zéro à un modèle qui tourne

---

Vous avez `diabetes.csv` : 768 lignes, 8 features médicales, une cible binaire (diabétique ou non). Hier vous avez codé le forward pass colonne par colonne, calculé les gradients à la main, implémenté Adam ligne par ligne. Aujourd'hui, ces 50 lignes de numpy vont tenir en 10 lignes de Keras. Mais avant d'écrire ces 10 lignes, on regarde ce qu'elles cachent : parce que si vous ne savez pas ce que `model.fit()` fait sous le capot, vous serez bloqué dès que quelque chose ne se comporte pas comme prévu.

### L'essentiel

#### Setup de l'environnement

[TensorFlow](#) est le framework de calcul numérique de Google. [Keras](#) en est l'interface haut niveau, incluse nativement depuis TF 2.x.

```
pip install tensorflow keras-tuner
python -c "import tensorflow as tf; print(tf.__version__);
print(tf.config.list_physical_devices('GPU'))"
```

Si la liste GPU est vide, l'entraînement tourne sur CPU : acceptable pour J2 (pas de CNN, datasets petits). Pour J3-J4-J5, on bascule sur [Google Colab](#) qui fournit un GPU T4 gratuitement.

Un point pratique : quand vous installez TensorFlow, vous avez le choix entre `tensorflow` (avec support GPU, ~600 Mo) et `tensorflow-cpu` (CPU uniquement, ~300 Mo). En développement local sans GPU, `tensorflow-cpu` suffit et évite de télécharger les bibliothèques CUDA. Sur Colab, pas de question : `tensorflow` est déjà installé avec le GPU activé.

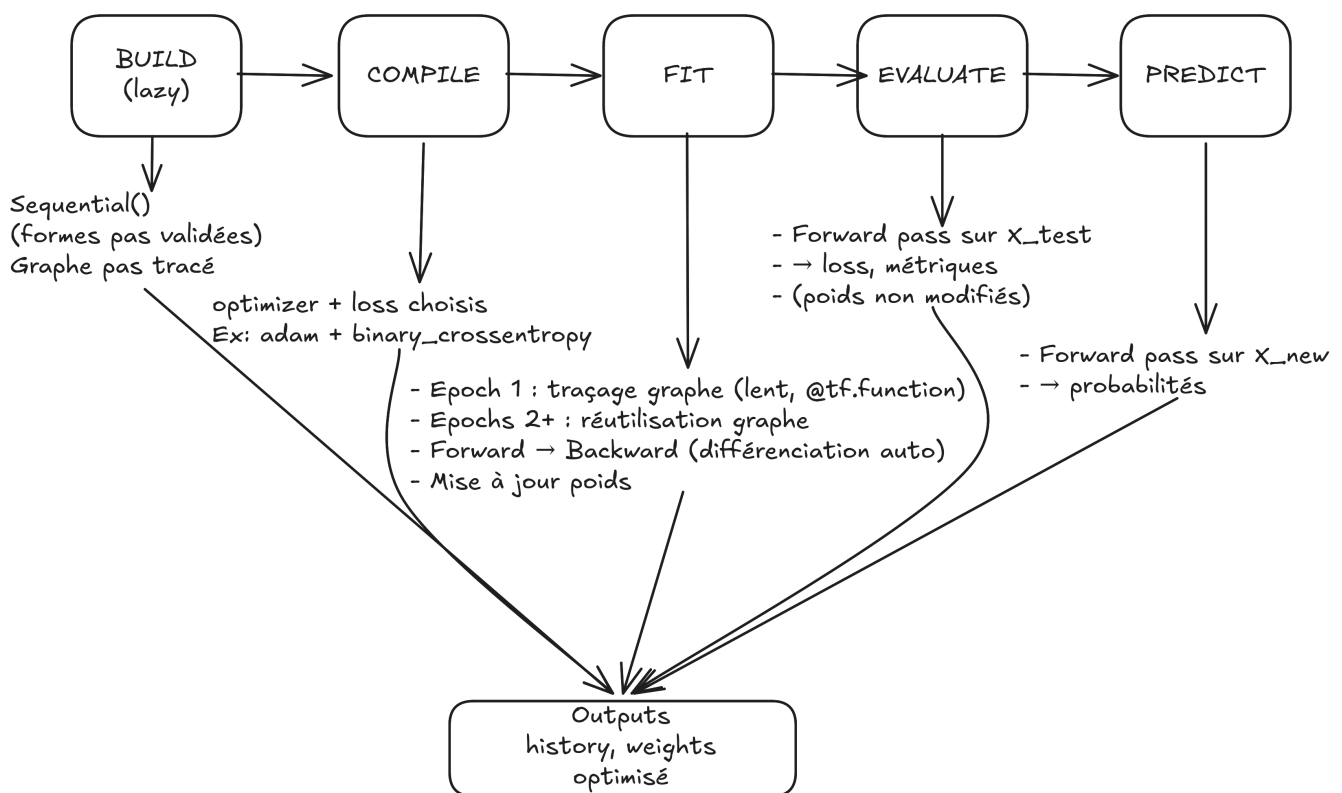
## La carte du trajet : les 5 étapes d'un projet Keras

Avant de plonger dans le code, voici le plan. Un projet Keras, quelle que soit la tâche, suit toujours les mêmes cinq étapes, dans cet ordre :

- **model** : déclarer l'architecture, c'est-à-dire la liste des couches (c'est le rôle de `Sequential`, juste en dessous).
- **compile** : choisir l'optimiseur, la loss et les métriques (comment le modèle apprend et comment on le juge).
- **fit** : entraîner le modèle sur les données d'entraînement.
- **evaluate** : mesurer la performance sur des données que le modèle n'a jamais vues.
- **predict** : se servir du modèle entraîné sur de nouveaux exemples.

Toute la section déroule ce pipeline case par case. Gardez le schéma en tête : chaque sous-section qui suit correspond à une de ces étapes.

### KERAS TRAINING PIPELINE



### `keras.Sequential` : votre réseau d'hier, déclaré au lieu d'être codé à la main

Hier, votre réseau c'était des matrices de poids empilées, des produits matriciels enchaînés et une activation appliquée à chaque couche, le tout écrit à la main en numpy. `keras.Sequential`, c'est le même réseau, mais *déclaré* : vous listez les couches dans l'ordre, et Keras se charge de créer les poids, d'enchaîner les produits matriciels et d'appliquer les activations.

Le mot "Sequential" dit tout : les couches s'enchaînent en file, la sortie de l'une devient l'entrée de la suivante. C'est le forward pass d'hier, en trois lignes au lieu de cinquante.

À quoi ça sert concrètement : à définir l'architecture (combien de couches, combien de neurones, quelles activations) sans jamais toucher aux poids ni aux gradients. Une fois le modèle déclaré, on l'entraîne avec `fit()` et on s'en sert avec `predict()` (on détaille ces étapes juste après).

**Tenseur ?** Tableau multidimensionnel de nombres. Un vecteur (1D), une matrice (2D), un batch d'images (4D : batch × hauteur × largeur × canaux). En DL, tout ce qui circule dans un réseau est un tenseur.

Voici un modèle déclaré avec Sequential :

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(8,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid') # sortie binaire
])
```

Une subtilité utile : Keras ne construit pas le graphe de calcul tout de suite. Il attend le premier appel réel (le premier `fit()` ou `predict()`) pour le "tracer", vérifier les shapes et optimiser la mémoire. C'est la construction "lazy", à la demande.

**Sous le capot (bon à savoir, pas à retenir par coeur) :** par défaut, TF exécute chaque opération immédiatement, comme du Python normal (mode "eager"). Pendant `fit()`, Keras bascule tout seul en mode "graph" via `@tf.function` : il fige les opérations en un graphe qu'il exécute en C++ optimisé, bien plus rapide, en s'appuyant sur des routines de calcul matriciel ultra-optimisées (les mêmes que NumPy utilise déjà sous le capot). Vous n'avez rien à coder pour ça. La seule chose à retenir : le tout premier epoch est plus lent (il paie le traçage du graphe une fois), les suivants sont rapides.

La limite du Sequential : il exige un flux linéaire, une entrée, une sortie. Dès que votre architecture a plusieurs entrées, des connexions qui sautent des couches (skip connections), ou des branches parallèles, Sequential ne suffit plus. On voit ça dans "Pour les curieux".

**He init, c'est quoi ?** Initialisation des poids proposée par Kaiming He (Microsoft Research, 2015). Pour ReLU, initialiser les poids avec une variance de `2/n_entrees` évite que le signal s'effondre ou explose au passage des couches profondes. Keras l'applique automatiquement sur les couches Dense avec ReLU. Vous n'avez pas à le spécifier, mais si vous voyez `kernel_initializer='he_normal'` dans un code, c'est ça.

**`model.summary()` : décortiquer les chiffres**

```
model.summary()
```

Model: "sequential"

| Layer (type)    | Output Shape | Param # |
|-----------------|--------------|---------|
| dense (Dense)   | (None, 64)   | 576     |
| dense_1 (Dense) | (None, 32)   | 2080    |
| dense_2 (Dense) | (None, 1)    | 33      |

=====  
Total params: 2,689  
Trainable params: 2,689  
Non-trainable params: 0

Chaque ligne se calcule avec `entrées × sorties + sorties` (les "+sorties" c'est le vecteur de biais, un par neurone de sortie) :

- Couche 1 : 8 features en entrée × 64 neurones + 64 biais = **576**
- Couche 2 : 64 × 32 + 32 = **2 080**
- Couche 3 : 32 × 1 + 1 = **33**

Total : 2 689 paramètres entraînaables. C'est raisonnable pour 768 exemples.

Model: "sequential"

| Layer (type)    | Output Shape | Param # |
|-----------------|--------------|---------|
| dense (Dense)   | (None, 64)   | 576     |
| dense_1 (Dense) | (None, 32)   | 2080    |
| dense_2 (Dense) | (None, 1)    | 33      |

=====  
Total params: 2,689  
Trainable params: 2,689  
Non-trainable params: 0

La règle de suspicion : au-delà de ~500 000 paramètres sans dataset énorme (>50k exemples bien diversifiés), le réseau va mémoriser les données d'entraînement au lieu d'apprendre à généraliser. C'est l'**overfitting par capacité excessive**. `model.summary()` est votre premier check systématique avant tout entraînement.

`(None, 64)` dans la colonne Output Shape : le `None` représente la dimension batch, inconnue à la définition du modèle (vous pouvez passer 1 exemple ou 256 exemples, ça ne change pas l'architecture).

## Avant d'entraîner : le vocabulaire qu'on va utiliser

On va faire deux choses avec nos données : **entraîner** un modèle dessus, puis **vérifier** qu'il a appris quelque chose d'utile (et pas juste récité par coeur). Chacune de ces deux étapes a son vocabulaire. On pose les mots maintenant, comme ça le code qui suit se lit sans buter dessus.

*Le déroulé EN BREF : on coupe les données en deux, on les normalise, on entraîne le modèle par petits paquets pendant plusieurs passages, et on mesure la performance au fur et à mesure.*

### Instant Notions : le vocabulaire de l'entraînement

- **Jeu d'entraînement (train) vs jeu de test (test)** : on met de côté une partie des données (typiquement 20%) que le modèle ne verra JAMAIS pendant l'entraînement. On l'entraîne sur les 80% restants (le train), puis on mesure sa performance sur les 20% gardés au chaud (le test). Pourquoi ? Un modèle qui a mémorisé ses données d'entraînement fera 100% dessus mais se plantera sur du nouveau. Le test, ce sont les "questions jamais vues" qui disent s'il a vraiment compris ou juste récité.
- **Jeu de validation (validation)** : un troisième morceau, découpé dans le train, qu'on regarde à chaque passage pendant l'entraînement pour surveiller si le modèle commence à sur-apprendre. Le test, lui, on n'y touche qu'une seule fois, tout à la fin. (Keras peut découper la validation tout seul avec `validation_split=0.2`.)
- **Normalisation** : mettre toutes les features à la même échelle (moyenne 0, écart-type 1). Un réseau apprend mal si une feature va de 0 à 1 et une autre de 0 à 10000. On détaille juste après avec `StandardScaler`.
- **Epoch** : un passage complet sur tout le jeu d'entraînement. On entraîne en général sur plusieurs epochs (20, 50, 100...).
- **Batch (ou mini-batch)** : le modèle ne voit pas les exemples un par un ni tous d'un coup, mais par petits paquets (32 par défaut). Chaque paquet déclenche une mise à jour des poids.
- **Step** : une mise à jour des poids, donc un batch traité. Une epoch = (nombre d'exemples / taille du batch) steps.
- **Loss (fonction de coût)** : le chiffre que le modèle cherche à faire baisser pendant l'entraînement. Plus la prédiction est loin de la vérité, plus la loss monte. C'est la boussole de l'optimiseur.
- **Métrique** : le chiffre qu'on lit, nous, pour juger si ça marche (par exemple l'accuracy, le % de bonnes réponses). La loss guide la machine, la métrique nous parle à nous. Ce ne sont pas forcément le même chiffre.

Gardez cette liste sous les yeux : tout le code qui suit ne fait qu'enchaîner ces briques.

## Classification sur données synthétiques

À vos claviers, pour entraîner un premier modèle de A à Z sur un jeu de données jouet. Copiez-tapez chaque ligne, n'exécutez pas tout d'un bloc (faut qu'ça rentre).

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow import keras
from tensorflow.keras import layers

# Données synthétiques : 500 points en forme de lune, deux classes entremêlées
```

```
X, y = make_moons(n_samples=500, noise=0.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalisation : chaque feature centrée-réduite
# StandardScaler ajuste sur le train UNIQUEMENT, puis transforme train et test
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test) # même scaler, pas de fit() ici
```

**Pourquoi fit uniquement sur le train ?** Si on fittait le scaler sur train + test, le modèle "verrait" indirectement la distribution du test pendant le preprocessing. C'est une fuite de données (data leakage) : les résultats sur le test set deviendraient trop optimistes.

Ce code fait trois choses, dans l'ordre.

**Les données.** `make_moons` fabrique 500 points répartis en deux croissants qui s'entremêlent (nos deux classes). Imaginez deux bananes emboîtées : impossible de séparer les deux classes avec une simple droite, il faut une frontière courbe. C'est exactement là qu'un réseau de neurones sert à quelque chose, un modèle linéaire échouerait. Le `noise=0.2` ajoute du désordre pour que ce ne soit pas trop facile, et `random_state=42` garantit qu'on retombe sur les mêmes points à chaque exécution.

**La coupe train/test.** `train_test_split` met 80% des points (400) dans le train et 20% (100) dans le test, en mélangeant l'ordre au passage. Le modèle apprendra sur les 400, et on jugera sa vraie performance sur les 100 qu'il n'aura jamais vus (le train vs test qu'on a défini plus haut).

**La normalisation.** `StandardScaler` recentre chaque feature : il calcule la moyenne et l'écart-type de chaque colonne, puis remplace chaque valeur par  $(\text{valeur} - \text{moyenne}) / \text{écart-type}$ . Résultat : chaque feature tourne autour de 0, à peu près entre -3 et +3. Concrètement :

```
Avant normalisation : X_train ≈ [[0.5, 0.8], [-1.2, 0.3], [2.1, -0.5], ...]
Après normalisation : X_train ≈ [[0.3, 0.5], [-0.8, 0.1], [1.2, -0.3], ...] # centrés
                                autour de 0
```

Le détail qui compte, c'est `fit_transform` sur le train mais `transform` (sans re-fit) sur le test : on calcule moyenne et écart-type sur le train seul, puis on applique ces mêmes chiffres au test. Fitter le scaler sur le test aussi ferait fuiter l'info du test dans le preprocessing, c'est le data leakage du blockquote juste au-dessus. Un réseau apprend nettement plus vite sur des données ainsi centrées-réduites.

```
# Modèle
model = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(2,)),
    layers.Dense(8, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Compilation
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Entraînement
history = model.fit(X_train, y_train, epochs=20, batch_size=32,
                    validation_split=0.2, verbose=1)
```

```
# Évaluation sur le test set (une seule fois, à la fin)
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy:.4f}")

# Prédiction sur 5 exemples
preds = model.predict(X_test[:5])
print("Probabilités brutes:", preds.flatten())
print("Classes prédites:", (preds > 0.5).astype(int).flatten())
```

Devrait afficher :

```
Epoch 1/20
10/10 [=====] - 1s 12ms/step - loss: 0.6811 - accuracy: 0.5750 -
val_loss: 0.6624 - val_accuracy: 0.6000
Epoch 2/20
...
Epoch 20/20
10/10 [=====] - 0s 3ms/step - loss: 0.2134 - accuracy: 0.9125 -
val_loss: 0.2089 - val_accuracy: 0.9250

Test accuracy: 0.9100
Probabilités brutes: [0.9324 0.0412 0.8871 0.1203 0.9567]
Classes prédites: [1 0 1 0 1]
```

Les valeurs exactes varieront légèrement (initialisation aléatoire), mais l'accuracy doit être entre 0.88 et 0.95 après 20 epochs.

**Rappel (epoch / batch / step, cf. vocabulaire plus haut) :** ici 400 exemples de train, dont 20% mis de côté en validation (`validation_split=0.2`), donc 320 exemples réellement entraînés, soit `320 / 32 = 10` steps par epoch. C'est le `10/10` que Keras affiche au début de chaque ligne.

## Cheat sheet : quelle activation, quelle loss, quelle métrique ?

### Instant Cheat Sheet! : paramétrisations par tâche

| Tâche                             | Activation sortie            | Loss                                         | Métrique principale   |
|-----------------------------------|------------------------------|----------------------------------------------|-----------------------|
| Régression                        | <code>linear</code> (aucune) | <code>mse</code> OU <code>mae</code>         | <code>mae</code>      |
| Classification binaire            | <code>sigmoid</code>         | <code>binary_crossentropy</code>             | <code>accuracy</code> |
| Multiclass (labels one-hot)       | <code>softmax</code>         | <code>categorical_crossentropy</code>        | <code>accuracy</code> |
| Multiclass (labels entiers bruts) | <code>softmax</code>         | <code>sparse_categorical_crossentropy</code> | <code>accuracy</code> |

`sparse_categorical_crossentropy` est identique à `categorical_crossentropy` sauf qu'elle accepte les labels entiers (0, 1, 2...) directement, sans passer par `to_categorical()`. Pratique pour éviter une étape de preprocessing.

**Rappel :** la **loss** est ce que l'optimiseur minimise (interne, technique), la **métrique** est ce qu'on lit pour juger si ça marche (cf. vocabulaire plus haut). Le tableau ci-dessus donne la bonne loss à choisir pour chaque type de tâche.



## Les méthodes clés : ce qui se passe vraiment

### `compile()`

La loss n'est pas juste une chaîne comme `'binary_crossentropy'`. C'est en réalité une fonction call-able `f(y_true, y_pred)` que vous pouvez passer directement : `loss=keras.losses.BinaryCrossentropy()`. La forme string est un raccourci qui résout vers l'instance correspondante. Utiliser l'instance directe vous donne accès aux paramètres, par exemple `label_smoothing=0.1` pour éviter que le modèle sur-confiant pousse les probabilités vers 0 et 1 stricts.

L'optimizer, lui, porte de l'état. [Adam](#) (Adaptive Moment Estimation) maintient deux moments par paramètre : `m` (moyenne des gradients passés, premier moment) et `v` (variance des gradients passés, second moment). Ces vecteurs sont initialisés à 0 et mis à jour à chaque step. C'est pour ça qu'Adam "chauffe" sur les premiers steps : ses estimations de moment sont biaisées vers zéro tant qu'elles ne se sont pas stabilisées. Adam corrige ça avec un facteur de correction de biais, ce qui explique pourquoi il converge proprement même dès la première epoch.

**Et le learning rate ?** Scalaire qui contrôle l'amplitude du pas de descente. Trop grand : les poids oscillent, la loss explose. Trop petit : la convergence prend des centaines d'epochs. La valeur par défaut d'Adam est `0.001`. Règle de départ : commencer à `0.001`, diviser par 10 si la loss oscille, multiplier par 3 si elle converge trop lentement.

[à imaginer avec les pas]

Les métriques sont indépendantes de la loss. Vous pouvez optimiser `binary_crossentropy` et suivre `[Precision(), Recall()]` simultanément : ça ne change pas ce qui est optimisé, ça ajoute juste des observateurs.

### `fit()`

À chaque epoch, Keras mélange le dataset (shuffle implicite sauf si `shuffle=False`), puis itère sur les mini-batches dans l'ordre mélangé. Sur chaque mini-batch : forward pass vectorisé sur le tenseur du batch, calcul de la loss, backward pass (différentiation automatique via le graphe TF), mise à jour des poids par Adam.

`validation_data=(X_val, y_val)` prend la priorité sur `validation_split` si vous passez les deux. Utilisez `validation_split` pour la plupart des cas; passez `validation_data` explicitement si vous avez normalisé vos données séparément (ce qui est la bonne pratique : fitter le scaler sur le train, transformer val et test séparément).

`fit()` retourne un objet `History`. Seul son attribut `.history` compte : c'est un dict avec une clé par métrique trackée, chaque valeur étant une liste de longueur `epochs`. C'est votre matière brute pour le diagnostic.

```
print(history.history.keys())
# dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
print(len(history.history['val_loss'])) # == epochs
```

### `evaluate()`

Retourne `[loss, metrique_1, metrique_2, ...]` dans l'ordre de compilation. À appeler une seule fois, à la fin, après toutes les décisions de tuning. Chaque appel "regarde" le test set : si vous ajustez l'architecture après avoir vu le résultat, vous overfittez le test set lui-même sans le savoir.

`predict()` VS `model(X, training=False)`

Voici une bizarrerie qui surprend tout le monde la première fois : le même modèle, sur les mêmes données, peut renvoyer deux prédictions différentes à deux appels de suite. Ce n'est pas un bug. Ça arrive dès que le modèle contient une couche **Dropout**.

**Dropout ?** Couche de régularisation qui éteint au hasard une partie des neurones (par exemple 50%) à chaque passage, mais UNIQUEMENT pendant l'entraînement. Ça force le réseau à ne pas dépendre d'un neurone précis, donc à moins sur-apprendre. On y revient en détail en section 5. Ce qui compte ici : pendant l'entraînement Dropout est actif, mais au moment de prédire pour de vrai il doit être désactivé (on veut tous les neurones, et un résultat stable). Doc : [Dropout](#).

D'où deux façons d'appeler le modèle, qui ne font PAS la même chose :

- `model.predict(X)` : **mode inférence**. Dropout désactivé, gère les batches tout seul, résultat déterministe (même entrée → même sortie). C'est ce qu'on utilise en production.
- `model(X, training=True)` : **mode entraînement forcé**. Dropout actif, donc chaque appel éteint d'autres neurones et donne une sortie différente. Utile pour un cas précis, le MC Dropout (estimer l'incertitude en faisant 50 appels et en regardant la variance des sorties), mais ce n'est pas ce qu'on veut pour une prédiction normale.

Vérifions-le tout de suite :

```
from tensorflow.keras import layers
import numpy as np

# Modèle avec Dropout
model_drop = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(2,)),
    layers.Dropout(0.5), # éteint 50% des neurones à chaque forward pass en training
    layers.Dense(1, activation='sigmoid')
])
model_drop.compile(optimizer='adam', loss='binary_crossentropy')
model_drop.fit(X_train, y_train, epochs=5, verbose=0)

X_sample = X_test[:3]

# predict() : déterministe, Dropout désactivé
pred_a = model_drop.predict(X_sample, verbose=0)
pred_b = model_drop.predict(X_sample, verbose=0)
print("predict() répété deux fois :", np.allclose(pred_a, pred_b)) # True

# model(X, training=True) : stochastique, Dropout actif
pred_c = model_drop(X_sample, training=True).numpy()
pred_d = model_drop(X_sample, training=True).numpy()
print("call training=True répété deux fois :", np.allclose(pred_c, pred_d)) # False
```

Devrait afficher :

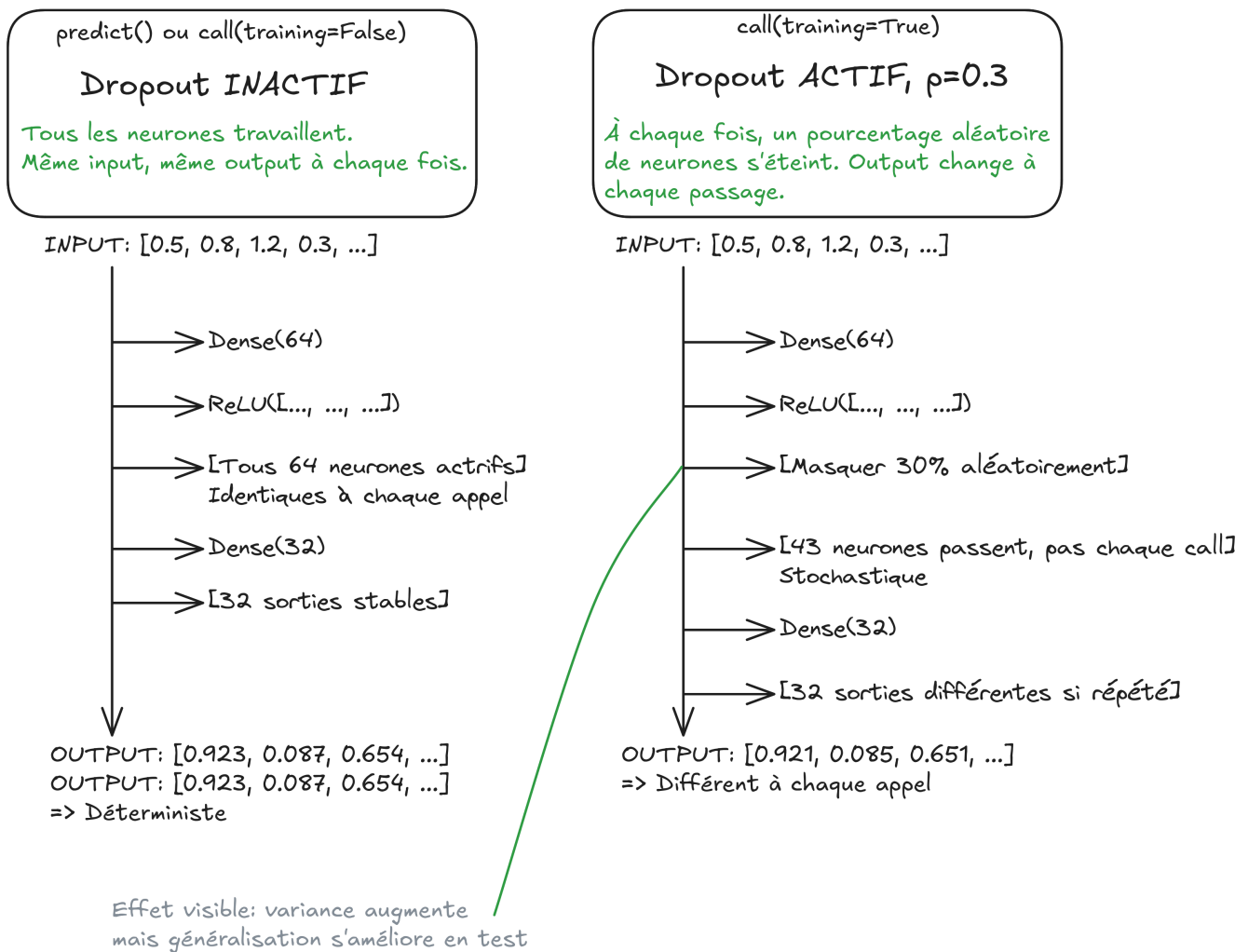
```
predict() répété deux fois : True
call training=True répété deux fois : False
```

- **pred\_a** et **pred\_b** (deux `predict()`) : résultats identiques, Dropout désactivé, comportement déterministe → `np.allclose(pred_a, pred_b)` renvoie `True`.
- **pred\_c** et **pred\_d** (deux `model(..., training=True)`) : résultats différents, Dropout actif, comportement stochastique → `np.allclose(pred_c, pred_d)` renvoie `False`.

Envie de voir les valeurs elles-mêmes ? Un simple `print(pred_a), print(pred_c)` ... affiche les probabilités : identiques pour les deux `predict()`, différentes à chaque `training=True`.

Grosso modo, on veut apprendre à notre modèle à être plus généraliste :

### DROPOUT : TRAINING vs INFERENCE



### `history.history` : votre instrument de bord

Le dictionnaire retourné par `fit()` est votre première source de diagnostic. Chaque clé est une liste de longueur `epochs` :

```
# {'loss': [...], 'accuracy': [...], 'val_loss': [...], 'val_accuracy': [...]}
```

```
# 20 epochs = 20 valeurs par clé

import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(history.history['loss'], label='train')
ax1.plot(history.history['val_loss'], label='validation')
ax1.set_title('Loss')
ax1.legend()

ax2.plot(history.history['accuracy'], label='train')
ax2.plot(history.history['val_accuracy'], label='validation')
ax2.set_title('Accuracy')
ax2.legend()

plt.tight_layout()
plt.show()
```

Ce qu'on cherche sur ces courbes : train et val qui descendent ensemble (bon apprentissage), ou val qui repart à la hausse pendant que train continue de descendre (overfitting). On le voit immédiatement, epoch par epoch. Sans `history.history`, on lancerait un `fit()` de 100 epochs, on attendrait, et on n'aurait qu'un seul chiffre final. Avec l'historique, on voit exactement à quelle epoch le modèle a commencé à overfitter. La section 3 approfondit la lecture de ces courbes.

COMPUTATION GRAPH (tracé par @tf.function)

Nœuds = opérations, Arêtes = tenseurs (données)

FORWARD PASS  
(et backward fusionné)

INPUT

$X$ : (batch=32, features=8)

Dense(64)

$W_1$ : (8,64)

$b_1$ : (64,1)

Opération 1:  $Z_1 = X @ W_1 + b_1$   
Kernels GPU: une seule fusion

$Z_1$ : (32, 64)

ReLU

Opération 2:  $A_1 = \text{ReLU}(Z_1)$

$A_1$ : (32, 64)

Dense(32)

$W_2$ : (64,32)

Opération 3:  $Z_2 = A_1 @ W_2 + b_2$

$Z_2$ : (32, 32)

ReLU

Opération 4:  $A_2 = \text{ReLU}(Z_2)$

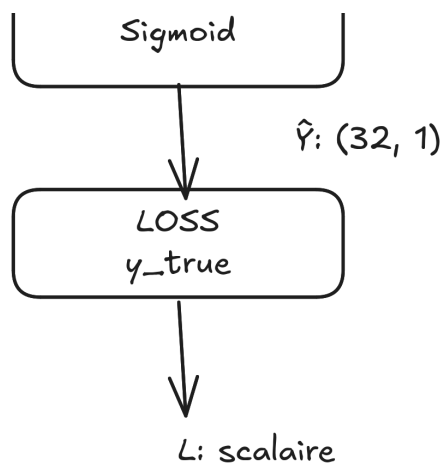
$A_2$ : (32, 32)

Dense(1)

$W_3$ : (32,1)

Opération 5:  $Z_3 = A_2 @ W_3 + b_3$

$Z_3$ : (32, 1)



Opération 6:  $Y = \text{Sigmoid}(Z_3)$

Opération 7:  $L = -\text{mean}(y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}))$

## Ce qui peut casser, et pourquoi

**Shapes incompatibles.** Vous définissez `input_shape=(8,)` mais votre `X_train` a 12 colonnes. Keras vous retourne `ValueError: Input 0 of layer dense is incompatible with the layer: expected axis -1 of input shape to have value 8, found 12`. Cause habituelle : un preprocessing qui a ajouté des colonnes (one-hot encoding d'une feature catégorielle, par exemple) sans mettre à jour `input_shape`. Réflexe : `print(X_train.shape)` avant de définir le modèle. Si vous êtes jamais certain, utilisez `input_shape=(X_train.shape[1],)` pour l'extraire dynamiquement.

**NaN dans la loss.** La loss devient `nan` ou `inf` au lieu de décroître. Trois causes dominant.

- Learning rate trop élevé : le réseau fait des pas tellement grands que les poids explosent, la loss part à l'infini, et les gradients deviennent `nan` en cascade.
- Données non normalisées : des features à très grande échelle (latitude/longitude en dizaines, âge en centaines) créent des gradients déséquilibrés qui déstabilisent Adam.
- Un `NaN` caché dans le dataset : une seule valeur manquante non gérée corrompt tout le batch. Pour diagnostiquer : `np.isnan(X).any()` pour les NaN, `X.describe()` pour repérer les min/max aberrants.
- Première action : normalisez avec `StandardScaler` et vérifiez les valeurs manquantes avant de toucher au learning rate.

**Classe déséquilibrée non détectée.** Votre modèle atteint 90% d'accuracy. Vous regardez ce chiffre et vous êtes satisfait. Mais votre dataset contient 90% de classe 0 et 10% de classe 1 : un modèle qui prédit toujours 0 atteint exactement 90% d'accuracy sans rien apprendre. En production, il ratera 100% des exemples de la classe 1 (celle qui vous intéressait).

-> La solution : toujours afficher `pd.Series(y).value_counts()` avant d'interpréter l'accuracy. Pour les datasets déséquilibrés, regardez la précision et le rappel par classe, ou passez `class_weight='balanced'` à `model.fit()` pour pénaliser davantage les erreurs sur la classe minoritaire.

"Mais du coup, le `model.fit()`, c'est quoi exactement dedans ? Y a la backprop ?"

Oui, et c'est exactement ce qu'on a codé hier. À chaque mini-batch : les données traversent le réseau couche par couche (forward pass), la loss mesure l'écart avec les vraies étiquettes, puis la chaîne de dérivées remonte à rebours couche par couche pour calculer le gradient de chaque poids (backward pass, c'est la chain rule qu'on a déroulée à la main hier). Adam ajuste les poids d'un petit pas dans la direction opposée au gradient. Keras fait tout ça en C++ optimisé, sur GPU si disponible. Même mathématique, 100x plus vite, et lisible en deux lignes.

Au fait on a quoi d'autre comme optimizers ? [En voilà ici](#) si on souhaite aller plus loin / étendre.

## Pour les curieux

### API fonctionnelle et skip connections

L'API Sequential exige un flux linéaire. Quand votre architecture a besoin de connexions qui sautent des couches (skip connections), de plusieurs entrées, ou de branches parallèles, il faut passer à l'[API fonctionnelle Keras](#).

#### Sequential (flux linéaire)

```
Input → Dense(64) → Dense(32) → Dense(1) → Output
```

#### Functional API avec skip connection

```
Input → Dense(64) → Dense(64) ┐  
└──────────────────────────┘ Add → Dense(1) → Output  
└ (shortcut)
```

Une skip connection ("connexion de raccourci") relie directement une couche à une couche plusieurs niveaux plus loin. Ça permet au gradient de "sauter" les couches intermédiaires pendant la backprop, ce qui résout un problème classique des réseaux profonds : les gradients deviennent minuscules en remontant et les premières couches n'apprennent presque rien. C'est le principe qui a rendu les [ResNets](#) (Deep Residual Networks, He et al. 2015) opérationnels à 50-100+ couches.

```
# Skip connection simple avec l'API fonctionnelle
inputs = keras.Input(shape=(64,))
x = layers.Dense(64, activation='relu')(inputs)
shortcut = inputs # on garde une référence à l'entrée

x = layers.Dense(64, activation='relu')(x)
x = layers.Add()([x, shortcut]) # addition tenseur : même shape obligatoire
outputs = layers.Dense(1, activation='sigmoid')(x)

model_functional = keras.Model(inputs=inputs, outputs=outputs)
model_functional.summary()
```

Si les shapes ne correspondent pas entre le shortcut et la sortie de la branche principale, on ajoute une projection : `shortcut = layers.Dense(64)(inputs)` pour adapter la dimension.

Pour J2, l'API Sequential couvre tout ce dont on a besoin. L'API fonctionnelle devient centrale en J4 (CNN avec ResNet-style blocks) et J5 (architectures multimodales).

### Custom training loop avec `tf.GradientTape`

Keras gère la boucle d'entraînement pour vous via `fit()`. Mais parfois un projet nécessite un training step non-standard : loss multi-composante, gradient clipping personnalisé, deux optimiseurs distincts (comme dans les GANs). Pour ça, on descend d'un niveau et on écrit la boucle à la main avec [tf.GradientTape](#).

```
import tensorflow as tf
```

```

optimizer = keras.optimizers.Adam(learning_rate=0.001)
loss_fn = keras.losses.BinaryCrossentropy()

# Boucle manuelle sur 5 epochs (simplifié, sans batching)
for epoch in range(5):
    with tf.GradientTape() as tape:
        # Tout ce qui se passe dans ce bloc est "tracé" : TF enregistre les opérations
        y_pred = model(X_train, training=True)      # forward pass
        loss = loss_fn(y_train, y_pred)             # calcul de la loss

        # tape.gradient() calcule les dérivées partielles de loss par rapport à chaque poids
        gradients = tape.gradient(loss, model.trainable_variables)
        # Appliquer les gradients : mise à jour des poids
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))
    print(f"Epoch {epoch+1}, loss: {loss.numpy():.4f}")

```

`GradientTape` enregistre chaque opération TF dans son contexte. Quand on appelle `tape.gradient()`, il remonte le graphe pour calculer les dérivées automatiquement : c'est la différentiation automatique (autograd). La différence avec `fit()` : ici vous contrôlez exactement ce qui se passe entre la forward pass et la mise à jour des poids. En J5, si votre groupe implémente un training step avec deux pertes combinées (reconstruction + classification), `GradientTape` sera votre outil.

## Mode eager et mode graph : le piège du debugging

En mode eager (le mode par défaut), `print(tensor)` affiche la valeur :

```

x = tf.constant([1.0, 2.0, 3.0])
print(x) # tf.Tensor([1. 2. 3.], shape=(3,), dtype=float32) -- fonctionne

```

À l'intérieur d'une fonction décorée avec `@tf.function`, TF trace le graphe : le code Python s'exécute une seule fois pour construire le graphe, puis le graphe est exécuté en C++ à chaque appel. Un `print()` Python s'exécute lors du traçage initial, pas à chaque appel. Si vous voulez afficher une valeur à chaque execution du graphe, utilisez `tf.print()` :

```

@tf.function
def forward(x):
    print("Ceci s'affiche seulement au traçage (une fois)") # traçage Python
    tf.print("Ceci s'affiche à chaque appel du graphe :", x) # exécution C++
    return x * 2

forward(tf.constant([1.0]))
forward(tf.constant([2.0]))

# Output:
# Ceci s'affiche seulement au traçage (une fois)
# Ceci s'affiche à chaque appel du graphe : [1]
# Ceci s'affiche à chaque appel du graphe : [2]

```

En pratique : pour déboguer pendant `fit()`, ne comptez pas sur les `print()` dans vos layers. Utilisez des callbacks ou le mode eager (retirez `@tf.function` temporairement, ou forcez `tf.config.run_functions_eagerly(True)`).



On sait construire un modèle, le compiler et l'entraîner. Mais "ça tourne" n'est pas "ça apprend bien" : un modèle peut terminer son entraînement sans jamais avoir réellement généralisé, ou s'être arrêté trop tôt. La prochaine étape, c'est d'apprendre à lire ce qui s'est vraiment passé pendant ces epochs : overfitting, underfitting, learning rate instable. Pour ça, on branche TensorBoard et on analyse les courbes en temps réel.

## 3 - Diagnostic : courbes loss/accuracy, TensorBoard, overfitting vs underfitting

Vous lancez `model.fit(epochs=100)`. Cinq minutes passent. La cellule affiche `val_accuracy: 0.85`. C'est bien ? C'est pas terrible ? Pour répondre honnêtement, un seul chiffre ne suffit pas. Ce chiffre final, c'est comme regarder la dernière image d'un film : vous avez manqué tout ce qui s'est passé avant. Ce qu'il vous faut, c'est le film entier. En pratique : les courbes epoch par epoch, et un outil pour les lire proprement.

### L'essentiel

#### Lire les courbes d'entraînement

`history.history` contient toutes les métriques epoch par epoch. C'est la sortie brute de `model.fit()`, un dictionnaire avec une liste par métrique : `{'loss': [...], 'accuracy': [...], 'val_loss': [...], 'val_accuracy': [...]}`. Cinquante epochs = cinquante valeurs dans chaque liste. Cette matière première, c'est ce qu'on trace.

```
import matplotlib.pyplot as plt

# history est la valeur de retour de model.fit()
history = model.fit(X_train, y_train, epochs=100,
                    validation_split=0.2, batch_size=32)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Courbe de loss : on suit la descente de l'erreur
axes[0].plot(history.history['loss'], label='train_loss')
axes[0].plot(history.history['val_loss'], label='val_loss')
axes[0].set_title('Loss')
axes[0].legend()

# Courbe d'accuracy : on suit la progression sur la tâche
axes[1].plot(history.history['accuracy'], label='train_accuracy')
axes[1].plot(history.history['val_accuracy'], label='val_accuracy')
axes[1].set_title('Accuracy')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Une fois les courbes tracées, on fait cinq lectures dans l'ordre :

- **Convergence** : est-ce que la `train_loss` descend du tout ? Si non, le modèle ne s'entraîne pas (bug de normalisation, learning rate trop bas, architecture cassée).

- **Écart train/val** : les deux courbes sont-elles proches ou largement séparées ? Un grand écart, c'est le signal d'alerte principal.
- **Stagnation** : les courbes s'aplatissent-elles trop tôt ? Si la `val_loss` ne bouge plus après 15 epochs sur 100, il y a quelque chose à changer.
- **Oscillations** : est-ce que la `val_loss` fait des spikes répétés au lieu d'une descente régulière ? C'est le symptôme d'un learning rate trop élevé, pas d'un overfitting.
- **Asymétrie loss/accuracy** : la loss descend mais l'accuracy stagne ? Ou inversement ? Les deux ne racontent pas toujours la même histoire (on creuse ça plus bas).

## Les quatre scénarios à reconnaître

**Scénario 1 : bon apprentissage.** `train_loss` et `val_loss` descendent ensemble, l'écart reste faible jusqu'à convergence. Le modèle a appris quelque chose de généralisable. Ce qu'on fait : lancer `model.evaluate()` sur le test set, noter le résultat, et passer au tuning si on veut gratter des points.

**Scénario 2 : overfitting (surapprentissage).** `train_loss` continue de descendre mais `val_loss` remonte ou stagne après un certain nombre d'epochs. L'écart entre les deux s'agrandit à chaque epoch. Le modèle mémorise le train set au lieu d'en extraire des patterns généralisables.

*Pourquoi ça arrive ?* Trop de capacité pour la quantité de données (réseau trop grand par rapport au dataset), pas de régularisation, entraînement trop long. C'est comme un élève qui apprend chaque question de l'examen par coeur. Sur les mêmes questions, 100%. Sur un sujet légèrement reformulé, il bloque.

Ce qu'on fait face à un overfitting : (1) repérer l'epoch où `val_loss` était minimale, (2) ajouter L2 ou Dropout (section 5), (3) essayer Early Stopping pour s'arrêter automatiquement au bon moment.

**Scénario 3 : underfitting (sous-apprentissage).** `train_loss` et `val_loss` restent élevées ensemble. L'écart est faible mais les deux valeurs absolues sont mauvaises. Le modèle n'a pas la capacité de capturer le signal dans les données.

Ce qu'on fait face à un underfitting : (1) vérifier le preprocessing (normalisation oubliée ?), (2) augmenter la capacité du réseau (plus de neurones, plus de couches), (3) vérifier que la fonction de perte est correcte pour la tâche (MSE pour une classification, c'est un bug, pas un underfit).

**Scénario 4 : oscillations fortes.** La `val_loss` ne converge pas proprement : elle descend, remonte, fait des spikes, et globalement ne va nulle part. `Train_loss` fait pareil ou diverge carrément vers l'infini.

Ce qu'on voit : `val_loss` qui remonte brutalement toutes les 3-4 epochs, ou une `train_loss` qui commence à remonter après quelques epochs. Ce n'est pas un problème de généralisation : c'est un problème de stabilité de l'optimisation. Learning rate trop élevé en première hypothèse. Le réseau fait des pas trop grands dans l'espace des poids et oscille autour du minimum au lieu d'y descendre. Ce qu'on fait : diviser le learning rate par 10 et relancer. `adam` avec `lr=0.001` (défaut) → essayer `lr=0.0001`.

## Loss vs accuracy : quand les deux ne racontent pas la même histoire

*Votre `val_accuracy` est à 78% depuis 20 epochs mais votre `val_loss` descend encore. C'est bon signe ou mauvais ?*

C'est un cas classique. L'accuracy et la loss mesurent des choses différentes. La loss (cross-entropy binaire, par exemple) mesure à quel point les probabilités prédites sont bien calibrées. L'accuracy ne regarde que si la prédiction finale est du bon côté du seuil 0.5. On peut avoir une loss qui descend (le modèle affine ses probabilités, de 0.52 à 0.61 pour la bonne classe) sans que l'accuracy bouge (0.52 > 0.5 et 0.61 > 0.5 donnent le même label).

Le cas qui trompe le plus souvent : un dataset déséquilibré. Imaginez 90% de classe 0 et 10% de classe 1. Un modèle qui prédit toujours 0 atteint 90% d'accuracy. Vous regardez ce chiffre et vous vous dites "c'est bien". Non. Le modèle n'a rien appris : il reproduit juste la fréquence de base des classes. En production, tous vos exemples de classe 1 (souvent la classe qui compte : fraude, maladie, défaut critique) passent à la trappe.

**AUC, c'est quoi ?** Area Under the ROC Curve. Mesure la capacité du modèle à séparer les classes, indépendamment du seuil. Score entre 0.5 (aléatoire) et 1.0 (parfait). Robuste aux déséquilibres de classes car elle évalue sur tous les seuils possibles en même temps. Sur un dataset à 90/10, un modèle qui prédit toujours 0 a une AUC de 0.5 (aléatoire), pas de 0.9. C'est pour ça qu'on la préfère à l'accuracy pour la détection de fraude, le diagnostic médical, les alertes de défaillance.

**F1, c'est quoi ?** Moyenne harmonique de la Précision et du Rappel. Précision = parmi tout ce que le modèle a dit "classe 1", combien étaient vraiment classe 1 ? Rappel = parmi tous les vrais exemples de classe 1, combien le modèle les a-t-il trouvés ?  $F1 = 2 * (Précision * Rappel) / (Précision + Rappel)$ . Score entre 0 et 1. Un F1 de 0.85 est un bon signal équilibré. Un F1 de 0.3 sur la classe minoritaire alors que l'accuracy est à 0.92, c'est le signe d'un modèle qui ignore la classe qui compte.

Règle pratique : sur un dataset équilibré, accuracy suffit pour suivre l'entraînement. Sur un dataset déséquilibré, suivez `val_loss` + `val_AUC` ou `val_F1`. L'accuracy reste affichée pour l'intuition mais ne décide rien.

```
# Ajouter AUC et précision/rappel comme métriques de suivi
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=[
        'accuracy',
        keras.metrics.AUC(name='auc'),          # robuste aux déséquilibres
        keras.metrics.Precision(name='precision'), # taux de faux positifs
        keras.metrics.Recall(name='recall')      # taux de faux négatifs
    ]
)
```

## TensorBoard : votre cockpit d'entraînement

[TensorBoard](#) est l'outil de visualisation officiel TensorFlow/Keras : interface web qui lit les logs produits pendant l'entraînement et les transforme en graphes interactifs. En équipe ML, c'est le standard pour comparer des runs et documenter des expériences.

Brancher TensorBoard : un callback passé à `model.fit()`, et un dossier de logs horodaté pour ne pas écraser les runs précédents.

```
import datetime

# Dossier unique par run : timestamp garantit qu'on n'écrase pas les logs précédents
log_dir = "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")

tensorboard_callback = keras.callbacks.TensorBoard(
    log_dir=log_dir,
    histogram_freq=1 # logger les histogrammes de poids à chaque epoch
                    # (0 = désactivé, 1 = chaque epoch, N = toutes les N epochs)
)
```

```
model.fit(X_train, y_train,
          epochs=100,
          validation_split=0.2,
          callbacks=[tensorboard_callback])
```

Lancer TensorBoard dans un terminal :

```
tensorboard --logdir=logs/fit
# Ouvrir http://localhost:6006 dans le navigateur
```

Sur Colab, c'est encore plus simple :

```
# Charger l'extension TensorBoard dans le notebook
%load_ext tensorboard
%tensorboard --logdir logs/fit
# L'interface apparaît directement dans la cellule
```

La documentation officielle pour Colab : [TensorBoard in Colab](#).

### Les onglets principaux, un par un.

**Scalars** : le panneau de bord principal. Toutes les métriques que Keras loggue automatiquement (loss, val\_loss, accuracy, val\_accuracy...) s'affichent sous forme de courbes. L'onglet permet de :

- Comparer plusieurs runs sur le même graphe (cocher/décocher dans le panneau de gauche)
- Utiliser le slider "Smoothing" pour lisser les courbes bruyantes : un EMA (Exponential Moving Average) qui atténue les spikes sans masquer les tendances. Un smoothing à 0.6-0.8 est souvent utile pour lire les tendances sur des runs qui oscillent.
- Filtrer les tags (les noms des métriques) si vous en avez beaucoup
- Télécharger les données CSV pour les analyser ailleurs

**Histograms** : distribution des poids et des activations par epoch. C'est ici qu'on voit ce que matplotlib ne montre pas. Chaque couche a son histogramme : si tous les poids d'une couche convergent vers zéro, cette couche ne contribue plus rien. Cas typiques à repérer :

- Dying ReLU : les histogrammes d'une couche ReLU s'effondrent vers zéro epoch après epoch. La couche est "morte" : elle ne propage plus aucun gradient. Solution : LeakyReLU, ou vérifier la normalisation des entrées.
- Gradient explosion : les histogrammes s'étalent de plus en plus loin de zéro, les queues grossissent à chaque epoch. Les poids dérivent vers des valeurs extrêmes. Solution : gradient clipping ou réduction du learning rate.

**Graphs** : visualisation de l'architecture du modèle. Sur un modèle Sequential, c'est un empilement de boîtes. Sur l'API fonctionnelle (qu'on voit en section 4 et sur laquelle on revient en J5), c'est le graphe complet avec les connexions. Utile pour debugger une architecture complexe : vérifier que les couches sont branchées dans le bon ordre, repérer une connexion manquante.

**Profiler** : analyse de performance step par step. Montre le temps passé dans chaque opération (forward pass, backward pass, transfert mémoire CPU/GPU). Utile à partir de J3-J4 quand on travaille sur des datasets plus lourds (CNN sur images, LSTM sur séries). On y revient.

**Projector** : visualisation des embeddings (représentations vectorielles). Permet de projeter en 2D/3D des couches de type `Embedding` pour voir si des clusters sémantiques se forment. On y revient en J4 sur la partie NLP.

### Deux runs côte à côte.

On entraîne deux modèles sur California Housing : un sans normalisation (qui diverge) et un avec normalisation (qui converge). L'objectif est de voir dans TensorBoard comment un problème de preprocessing se lit sur les courbes, avant même de regarder le score final.

```
# Run 1 : sans normalisation (les gradients vont diverger)
log_dir_bad = "logs/fit/run_sans_norm"
tb_bad = keras.callbacks.TensorBoard(log_dir=log_dir_bad, histogram_freq=1)

model_bad = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(8,)),
    layers.Dense(1)
])
model_bad.compile(optimizer='adam', loss='mse', metrics=['mae'])
model_bad.fit(X_train_raw, y_train, epochs=50, validation_split=0.2,
              callbacks=[tb_bad], verbose=0)

# Run 2 : avec normalisation (StandardScaler appliqué sur X_train)
log_dir_good = "logs/fit/run_avec_norm"
tb_good = keras.callbacks.TensorBoard(log_dir=log_dir_good, histogram_freq=1)

model_good = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(8,)),
    layers.Dense(1)
])
model_good.compile(optimizer='adam', loss='mse', metrics=['mae'])
model_good.fit(X_train_norm, y_train, epochs=50, validation_split=0.2,
               callbacks=[tb_good], verbose=0)
```

Devrait afficher (résumé après les deux `fit`) :

```
run_sans_norm : val_loss ~25-100 (explose ou oscille dès epoch 5-10)
run_avec_norm : val_loss descend proprement, converge vers ~0.3 en ~30 epochs
```

Dans TensorBoard, onglet Scalars : les deux runs apparaissent côte à côte avec des couleurs différentes. La `val_loss` du run sans normalisation explose ou reste instable dès les premières epochs. Celle du run normalisé descend proprement.

Dans l'onglet Histograms : les poids du run sans normalisation montrent des distributions qui s'étalent ou s'effondrent selon les couches. Le run normalisé garde des distributions stables.

**TensorBoard vs les autres outils.** [Weights & Biases](#) (W&B) est l'alternative cloud la plus utilisée en équipe : logs automatiques, comparaison de runs en ligne, partage de résultats avec un lien. [MLflow](#) est plutôt orienté cycle de vie du modèle : versioning des expériences, registry des modèles, déploiement. TensorBoard reste le choix naturel si vous travaillez en local sur TF/Keras et que vous n'avez pas besoin de collaboration.

## Ce qui peut tromper

Trois cas où les courbes induisent en erreur :

**Oscillations de la val\_loss.** Des spikes réguliers sur la val\_loss (montée brutale puis redescende, en boucle) : c'est un learning rate trop élevé, pas un overfitting. Le modèle oscille autour d'un minimum sans pouvoir s'y stabiliser. Action : réduire le learning rate d'un facteur 10.

**Val\_accuracy qui plafonne alors que val\_loss descend.** Souvent un signe de dataset déséquilibré ou d'un seuil de décision inadapté. Le modèle affine ses probabilités (val\_loss descend) mais le seuil à 0.5 reste arbitrairement mauvais. Action : regarder val\_AUC plutôt que val\_accuracy, et envisager de changer le seuil de décision selon le contexte métier.

**Train\_loss qui remonte en cours d'entraînement.** Le cas le moins intuitif : la train\_loss devrait toujours descendre ou stagner, jamais remonter. Si c'est le cas, c'est souvent un optimiseur instable (learning rate trop élevé avec Adam peut causer ça sur des architectures avec Batch Normalization), ou un bug de normalisation découvert tardivement (données de train et de val normalisées différemment). Avec `batch_size=1`, la Batch Normalization elle-même peut devenir instable : la statistique calculée sur un seul exemple est trop bruitée. Action : vérifier la normalisation, augmenter le batch size, ou passer à un scheduler de learning rate.

---

## Diagnostic rapide : checklist

- `train_loss` descend, `val_loss` monte : overfitting. Aller en section 5 (régularisation).
  - Les deux stagnent hautes : underfitting. Vérifier la normalisation en premier, puis augmenter la capacité.
  - Les deux descendent ensemble : bon signe. Regarder val\_AUC si le dataset est déséquilibré.
  - `val_loss` oscille avec des spikes réguliers : learning rate trop élevé. Diviser par 10.
  - `val_accuracy` stagne mais `val_loss` descend : regarder val\_AUC ou val\_F1, changer le seuil de décision.
  - `train_loss` remonte en cours d'entraînement : vérifier la normalisation, le batch\_size, et l'optimiseur.
- 

## Approfondissons

### TensorBoard HParams : comparer des configurations d'hyperparamètres

L'onglet `HParams` de TensorBoard permet de visualiser comment les hyperparamètres influencent les métriques finales, sur l'ensemble de vos runs de tuning. On l'utilise en Phase 6 de l'après-midi avec keras-tuner.

```

from tensorboard.plugins.hparams import api as hp

# Déclarer les hyperparamètres qu'on veut tracker
HP_UNITS = hp.HParam('units', hp.Discrete([32, 64, 128]))
HP_LR = hp.HParam('learning_rate', hp.RealInterval(1e-4, 1e-2))
METRIC_ACC = 'accuracy'

# Dans chaque run : logger les valeurs d'hyperparamètres + la métrique finale
with tf.summary.create_file_writer('logs/hparam_tuning').as_default():
    hp.hparams({'units': 64, 'learning_rate': 0.001})
    tf.summary.scalar(METRIC_ACC, value=0.82, step=1)

```

La vue "Parallel Coordinates" dans l'onglet HParams est particulièrement utile : chaque run est représenté par une ligne qui traverse plusieurs axes verticaux, un axe par hyperparamètre plus un axe pour la métrique objectif. Les lignes qui se croisent peu et arrivent dans le haut de l'axe accuracy sont les configurations gagnantes. Vous voyez immédiatement si, par exemple, toutes les configurations avec `learning_rate > 0.005` finissent en bas de l'accuracy, quel que soit `units`. Cela permet de réduire l'espace de recherche pour le run suivant.

## Logger des métriques custom avec tf.summary

Keras loggue automatiquement les métriques passées à `model.compile()`. Mais parfois on veut logger autre chose : une métrique calculée manuellement (calibration, précision par classe, score métier). Pour ça, `tf.summary.scalar()` dans un contexte `tf.summary.create_file_writer()`.

```

import tensorflow as tf

# Créer un writer pour le dossier de logs
writer = tf.summary.create_file_writer("logs/custom_metrics")

# Calculer des métriques custom après chaque epoch
for epoch in range(50):
    history_epoch = model.fit(X_train, y_train, epochs=1,
                              validation_split=0.2, verbose=0)

    # Calculer une métrique custom, ici la précision sur la classe 1 uniquement
    y_pred = (model.predict(X_val, verbose=0) > 0.5).astype(int)
    precision_class1 = precision_score(y_val, y_pred)

    # Logger dans TensorBoard
    with writer.as_default():
        tf.summary.scalar('precision_class_1', data=precision_class1, step=epoch)

```

Utile pour des cas où les métriques Keras ne suffisent pas : calibration d'un modèle de scoring, précision par classe sur Wine Quality, ou n'importe quelle métrique métier que Keras ne calcule pas nativement.

## Smoothing : ce que fait réellement le slider TensorBoard

Le slider "Smoothing" dans l'onglet Scalars applique un EMA (Exponential Moving Average) sur la courbe brute. La formule :  $\text{smoothed\_t} = \alpha * \text{smoothed\_}\{t-1\} + (1 - \alpha) * \text{raw\_t}$ . Avec  $\alpha = 0.6$  (défaut TensorBoard), chaque point lissé dépend à 60% du point précédent et à 40% de la valeur brute. Plus  $\alpha$  est élevé, plus la courbe est lisse mais plus elle est "en retard" sur les changements réels. Un smoothing à 0 donne la courbe brute. La règle pratique : utiliser le smoothing pour lire les tendances générales, mais toujours vérifier la courbe brute (smoothing = 0) pour confirmer qu'un spike n'est pas masqué.

 **Bonus perso** : Si vous êtes en alternance data, TensorBoard est un outil que vous pouvez proposer de mettre en place dès maintenant sur des entraînements sklearn via le plugin [tensorboard-plugin-profile](#). Mini-challenge hors-cours : brancher TensorBoard sur un entraînement Random Forest sklearn et mesurer le temps passé dans chaque étape. Ça fait un bon sujet de conversation en entretien technique.

On a maintenant les outils pour lire ce qui se passe pendant l'entraînement. Les courbes nous disent si le modèle apprend bien ou s'il déraile, et TensorBoard nous donne la même information avec beaucoup plus de puissance de comparaison. Mais "diagnostiquer" n'est que la première moitié du travail. La deuxième : **décider quoi changer, dans quel ordre, et jusqu'où aller**. C'est exactement le sujet de la section suivante : comment on cherche les bons hyperparamètres sans tester chaque combinaison à la main.

## 4 - Optimisation et tuning des hyperparamètres

Scénario concret pour ouvrir : votre PMC plafonne à 73% de `val_accuracy` sur le dataset Pima. Le modèle apprend, il n'overfitte pas, mais il stagne. *Vous touchez d'abord le learning rate, la taille des couches, ou le batch size ?* La plupart des gens répondent au hasard. Après cette section, vous aurez une réponse systématique.

### L'essentiel

#### Hyperparamètre vs paramètre : la distinction fondamentale

Un **paramètre** est une valeur que le modèle apprend pendant l'entraînement : les poids `w` et les biais `b`. Vous ne les fixez pas, la backpropagation les optimise.

Un **hyperparamètre** est une valeur que vous choisissez avant l'entraînement. Le modèle ne l'apprend pas : c'est votre décision d'ingénieur. Exemples : le learning rate, le nombre de couches, le nombre d'unités par couche, la taille du batch, le type d'optimiseur, le taux de dropout.

Le tuning des hyperparamètres consiste en quoi du coup ? => À **chercher**, dans l'espace des configurations possibles, **celle qui donne le meilleur modèle sur la validation**. Le problème : cet espace est énorme, et chaque évaluation prend du temps.

#### Dans quel ordre on touche quoi ?

Toutes les décisions d'ingénierie ne se valent pas. Voici l'ordre d'impact empirique, du plus fort au plus faible.

**1. Le learning rate (impact : majeur)**. C'est le levier le plus puissant.

- Trop élevé : la loss oscille ou diverge.
- Trop bas : le modèle apprend, mais très lentement et peut stagner dans un mauvais minimum local.



La plage typique à explorer : `[1e-4, 1e-3, 1e-2]`. Un facteur 10 dans le learning rate peut doubler la performance finale, ou tuer l'entraînement complètement. Commencez toujours par là.

**2. L'architecture (impact : majeur à fort).** La profondeur (nombre de couches) et la largeur (nombre d'unités par couche) définissent la capacité du modèle à représenter des fonctions complexes. Un modèle trop petit ne peut pas apprendre la structure du problème (underfitting). Un modèle trop grand mémorise le train set (overfitting). Sur des données structurées, tester des architectures dans des ordres de grandeur différents : 1 couche / 2 couches / 3 couches, puis 32 / 64 / 128 unités.

**3. Le batch size (impact : moyen sur la qualité, fort sur la vitesse).** Un batch size petit (16, 32) : plus de bruit dans le gradient, souvent meilleure généralisation, mais entraînement plus lent. Un batch size grand (256, 512) : gradients plus stables, entraînement rapide, mais risque de converger vers des minima "plats" qui généralisent moins bien. La règle de base : 32 ou 64 pour commencer. Ne touchez au batch size que si le learning rate et l'architecture sont déjà stables.

**4. La fonction d'activation (impact : faible une fois qu'on est sur ReLU).** Sur les couches cachées, ReLU par défaut. Passer de ReLU à ELU ou Leaky ReLU peut améliorer de 1 à 2 points dans des cas spécifiques (neurones morts). Mais changer d'activation sans avoir d'abord optimisé le learning rate et l'architecture, c'est du bruit.

**5. La régularisation (L2, Dropout) (impact : fort contre l'overfitting, neutre ou négatif si pas d'overfitting).** On y revient en section 5 en détail. L'ordre d'action : d'abord trouver la bonne architecture avec un bon learning rate, puis régulariser si l'overfitting apparaît.

La règle de décision : chaque fois que vous ne savez pas quoi toucher, commencez par le learning rate. Si la loss diverge ou oscille fortement, il est trop élevé. Si la loss descend très lentement et ne s'améliore plus après N epochs, il est trop bas (ou l'architecture est trop petite). Ce diagnostic vient du TensorBoard qu'on vient de voir.

## Grid Search : exhaustif mais coûteux

La [Grid Search](#) teste toutes les combinaisons d'une grille prédéfinie. Exemple :

```
3 valeurs de learning_rate × 3 tailles de couche cachée × 2 activations = 18 runs
```

18 runs × 50 epochs × 30 secondes par epoch : ça fait **4h30 de wall time**. Sur une grille 5×5×3 avec 50 epochs, on dépasse les 50 heures. La grid search explose combinatoirement : c'est son principal défaut.

Pour intégrer un modèle Keras dans le pipeline scikit-learn,

`tf.keras.wrappers.scikit_learn.KerasClassifier` était la solution historique, mais elle a été **dépréciée depuis TF 2.16**. La solution actuelle : [scikeras](#), un wrapper communautaire maintenu qui fait la même chose proprement.

```
pip install scikeras
```

```
from sklearn.model_selection import GridSearchCV
from scikeras.wrappers import KerasClassifier # remplace l'ancien wrapper déprécié

def create_model(units=64, learning_rate=0.001):
    model = keras.Sequential([
        layers.Dense(units, activation='relu', input_shape=(8,)),
```

```

        layers.Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

# KerasClassifier encapsule le modèle Keras dans l'interface scikit-learn
clf = KerasClassifier(model=create_model, epochs=50, verbose=0)

param_grid = {
    'model__units': [32, 64, 128],
    'model__learning_rate': [0.0001, 0.001, 0.01]
}

# cv=3 : 3-fold cross-validation sur chaque config (18 configs × 3 folds = 54 runs)
grid = GridSearchCV(clf, param_grid, cv=3, scoring='accuracy')
grid_result = grid.fit(X_train, y_train)
print("Meilleure config :", grid_result.best_params_)
print("Score CV :", grid_result.best_score_)

```

**scikeras, c'est quoi ?** Un wrapper qui permet d'utiliser un modèle Keras comme s'il était un estimateur scikit-learn. Ça débloquent GridSearchCV, RandomizedSearchCV, et n'importe quel pipeline sklearn sur vos modèles Keras.

La grid search est utile quand l'espace est petit et que vous voulez une couverture exhaustive. Dès que l'espace dépasse 4 ou 5 dimensions, elle devient inutilisable en pratique.

## Random Search : souvent meilleur à budget égal

La [Random Search](#) échantillonne aléatoirement des configurations dans l'espace. Contre-intuitif, mais : dans un espace multidimensionnel, certains hyperparamètres ont beaucoup d'impact (le learning rate) et d'autres peu (le batch size). La grid search gaspille des runs sur des combinaisons redondantes. La random search explore plus de valeurs distinctes pour les paramètres importants.

L'idée c'est qu'on cherche une zone intéressante dans un foin à 10 dimensions. La grid search quadrille le foin en cases régulières : pour chaque valeur du learning rate, elle teste les mêmes 3 valeurs de batch size, même si le batch size n'a aucun impact. La random search lance des sondes aléatoires : statistiquement, elle couvre plus de valeurs distinctes du learning rate avec le même budget de runs.

[Bergstra & Bengio \(2012\)](#) ont montré empiriquement sur de nombreux benchmarks ML que la random search bat la grid search à budget égal sur la quasi-totalité des problèmes. L'intuition : si la fonction "config -> perf" est surtout sensible à 2 paramètres sur 6, une random search de 60 configs explore 60 valeurs différentes de ces 2 paramètres. Une grid search  $3 \times 3 \times 3 \times 3 \times 3 \times 3$  n'en explore que 3.

En pratique avec scikeras :

```

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform # distribution log-uniforme pour les taux

# loguniform(1e-4, 1e-1) : échantillonne le learning rate entre 1e-4 et 1e-1

```

```
# sur une échelle logarithmique (ce qui a du sens : 1e-3 et 2e-3 sont proches,
# 1e-4 et 1e-3 sont aussi "proches" sur l'échelle log, mais séparés de 10x)
param_dist = {
    'model__units': [32, 64, 128, 256],
    'model__learning_rate': loguniform(1e-4, 1e-1)
}

random_search = RandomizedSearchCV(
    clf, param_dist,
    n_iter=20, # 20 configs aléatoires au lieu de toutes les combiner
    cv=3,
    scoring='accuracy',
    random_state=42 # reproduire les résultats
)

random_search.fit(X_train, y_train)
print("Meilleure config :", random_search.best_params_)
```

## Optimisation bayésienne : pas juste de la culture

L'optimisation bayésienne fait ce que ni la grid search ni la random search ne font : elle **apprend** de l'historique des runs pour choisir les configs suivantes.

Principe de fonctionnement :

1. Les premiers runs (10 à 20) sont essentiellement aléatoires : on construit une base de données de (config, performance).
2. Un **surrogate model** (modèle de substitution) est entraîné sur cet historique. Il apprend à approximer la fonction `f(config) → performance` attendue. En pratique, les surrogates utilisés sont des Processus Gaussiens ou des arbres de régression.
3. À chaque nouvelle itération, au lieu de choisir au hasard, on choisit la config qui maximise une **fonction d'acquisition**. Cette fonction balance deux objectifs : exploiter les zones déjà bonnes (exploitation) et explorer les zones inconnues (exploration).

**Fonction d'acquisition ?** Une formule qui mesure "à quel point cette config vaut la peine d'être testée". Les plus courantes : Expected Improvement (EI, combien je m'attends à gagner par rapport au meilleur run actuel) et Upper Confidence Bound (UCB, somme de la perf prédite + incertitude). Le choix entre EI et UCB est hors programme : en pratique Optuna choisit pour vous.

Quand l'optimisation bayésienne brille : quand chaque évaluation est chère (1h d'entraînement, instance GPU facturée), et quand l'espace d'hyperparamètres est continu (learning rate en `float`, pas une liste de 3 valeurs). Elle trouve de meilleures configs avec 3x à 5x moins de runs qu'une random search sur ces cas.

Caveat important : **les 10 à 20 premiers runs sont encore aléatoires** (bootstrap du surrogate). Sur un budget de 15 runs, l'optimisation bayésienne n'a pas vraiment le temps d'apprendre. Elle commence à briller à partir de 30 à 50 runs.

Bibliothèques : [Optuna](#) (la plus populaire, framework-agnostique, s'intègre avec Keras/PyTorch/sklearn en 10 lignes) et le backend `BayesianOptimization` de [keras-tuner](#).

## keras-tuner : la solution Keras officielle

[keras-tuner](#) est la bibliothèque officielle Keras pour le tuning d'hyperparamètres. Elle gère la recherche, les runs, les logs, et l'identification du meilleur trial. Elle s'intègre directement avec TensorBoard HPParams.

On définit un **HyperModel** : une fonction `build_model(hp)` qui reçoit un objet `hp` (HyperParameters) et l'utilise pour définir l'architecture. keras-tuner passe différentes valeurs de `hp` pour explorer l'espace.

```
import keras_tuner as kt

def build_pima_model(hp):
    model = keras.Sequential()

    # hp.Int : valeur entière entre min et max, par pas de `step`
    model.add(layers.Dense(
        units=hp.Int('units', min_value=32, max_value=256, step=32),
        activation='relu',
        input_shape=(8,)
    ))

    # Couche optionnelle : keras-tuner peut décider de l'inclure ou non
    if hp.Boolean('second_layer'):
        model.add(layers.Dense(
            units=hp.Int('units_2', min_value=16, max_value=128, step=16),
            activation='relu'
        ))

    model.add(layers.Dense(1, activation='sigmoid'))

    # hp.Choice : valeurs discrètes parmi une liste
    lr = hp.Choice('learning_rate', values=[1e-4, 5e-4, 1e-3, 5e-3, 1e-2])
    model.compile(
        optimizer=keras.optimizers.Adam(lr),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

# RandomSearch : max_trials configurations aléatoires dans l'espace défini par hp
tuner = kt.RandomSearch(
    build_pima_model,
    objective='val_accuracy',  # ce qu'on cherche à maximiser
    max_trials=20,             # nombre de configurations testées
    seed=42,                   # reproductibilité
    directory='tuning_logs',
    project_name='pima_tuning'
)

# verbose=1 pour voir les trials défiler en live
tuner.search(X_train, y_train, epochs=50, validation_split=0.2, verbose=1)
```

Devrait afficher (avec `verbose=1`):

```
Trial 1 Complete [00h 00m 23s]
val_accuracy: 0.7234

Best val_accuracy So Far: 0.7234
Total elapsed time: 00h 00m 23s

Trial 2 Complete [00h 00m 19s]
val_accuracy: 0.7468

Best val_accuracy So Far: 0.7468
Total elapsed time: 00h 00m 42s
...
```

Chaque trial est un run complet. À la fin :

```
# Récupérer le meilleur modèle directement prêt à l'évaluation
best_model = tuner.get_best_models(num_models=1)[0]
best_hp = tuner.get_best_hyperparameters()[0]

print("Meilleur lr :", best_hp.get('learning_rate'))
print("Meilleures units :", best_hp.get('units'))
print("Deuxième couche :", best_hp.get('second_layer'))
```

Les résultats sont sauvegardés dans `tuning_logs/pima_tuning/`. Pour les visualiser dans TensorBoard :

```
tensorboard --logdir tuning_logs/pima_tuning
```

Dans TensorBoard, onglet **HParams** : deux vues complémentaires.

- La vue "Parallel Coordinates" trace une ligne par trial à travers toutes les colonnes d'hyperparamètres, colorée par `val_accuracy`. Les lignes claires (meilleures configs) forment un faisceau : c'est visuellement immédiat de voir que "learning\_rate entre 1e-3 et 5e-3 tend à donner de meilleures perfs".
- La vue Scatter Plot croise chaque hyperparamètre avec la métrique objectif : si les points forment une bande claire à certaines valeurs, vous tenez quelque chose.

## Ce qui peut mal se passer

**Data leakage.** Si vous tunez sur le test set (ou si votre val set a été construit après normalisation globale), les hyperparamètres "apprennent" le test set. La performance mesurée est irréaliste. Règle : tuner sur la validation, évaluer sur le test **une seule fois** après avoir choisi la config finale.

**Val set overfitting après 100+ trials.** Avec assez de trials sur le même val set, vous pouvez finir par choisir une configuration qui a eu de la chance sur CE val set spécifique, pas une configuration réellement meilleure. Solution : utiliser un val set plus grand (>20% du dataset), ou passer à une cross-validation (k-fold) pour les runs de tuning si le budget le permet.

**Oubli du seed.** Même hyperparamètres, runs différents : les résultats varient à cause de l'initialisation aléatoire des poids. Pour comparer deux configs de façon honnête, fixer les seeds :

```
import numpy as np
import tensorflow as tf

np.random.seed(42)
tf.random.set_seed(42)
```

Sans ça, un trial peut sembler meilleur qu'un autre juste parce qu'il a été chanceux à l'initialisation. Avec des datasets petits comme Pima (768 samples), la variance entre runs est non négligeable.

**Stopping trop tôt sur une config large.** Une configuration avec `units=256` peut avoir besoin de plus d'epochs pour converger qu'une avec `units=32` : le modèle plus large a plus de poids à ajuster. Si vous coupez à 50 epochs pour tous les trials, les grandes architectures sont désavantagées. Solution : augmenter le budget d'epochs pour les runs de tuning, ou laisser keras-tuner gérer ça avec Hyperband (voir ci-dessous).

## Tableau de comparaison

| Méthode               | Quand l'utiliser                                        | Efficacité d'échantillonnage | Parallélisable | Débogage |
|-----------------------|---------------------------------------------------------|------------------------------|----------------|----------|
| Grid Search           | Espace petit (<20 combos), couverture exhaustive voulue | Faible (redondances)         | Oui            | Simple   |
| Random Search         | Espace moyen, budget limité, bonne première exploration | Bonne                        | Oui            | Simple   |
| Bayesian Optimization | Runs très coûteux (GPU, long training), espace continu  | Très bonne après 20+ runs    | Difficile      | Complexe |
| Hyperband             | Runs longs, budget temps fixé, recherche adaptative     | Très bonne                   | Oui            | Moyen    |

## On pousse plus loin

### Hyperband : la recherche adaptative

[Hyperband \(Li et al., 2016\)](#) résout un problème que la random search ne résout pas : comment allouer votre budget d'epochs intelligemment ?

Intuition : si vous lancez 64 configs au hasard et les entraînez 5 epochs chacune, vous pouvez déjà voir lesquelles sont prometteuses. Gardez les 1/3 meilleurs, donnez-leur plus d'epochs. Itérez. C'est le principe du **successive halving** : à chaque round, vous éliminez les configs les moins prometteuses et concentrez les ressources sur les survivantes.

Hyperband répète successive halving avec différents budgets initiaux (de petits budgets pour explorer large, de grands budgets pour affiner les meilleures configs). Résultat : sur un même budget total, Hyperband trouve souvent de meilleures configs que la random search pure, surtout quand l'entraînement est long.

```
# Hyperband en keras-tuner
tuner_hb = kt.Hyperband(
    build_pima_model,
    objective='val_accuracy',
    max_epochs=100,      # epochs max pour les configs qui survivent jusqu'au bout
    factor=3,            # facteur de réduction : 1/3 des configs avancent à chaque round
    seed=42,
    directory='tuning_logs',
    project_name='pima_hyperband'
)

tuner_hb.search(X_train, y_train, validation_split=0.2, verbose=1)
```

Sur Pima avec ses 768 samples, RandomSearch est souvent suffisant. Hyperband devient intéressant à partir du Jour 3 avec les CNN, où un run peut durer plusieurs minutes.

## Population-Based Training

[Population-Based Training \(Jaderberg et al., DeepMind 2017\)](#) pousse l'idée plus loin : au lieu de lancer des runs indépendants, PBT entraîne une population de modèles en parallèle et, pendant l'entraînement, les modèles moins performants "copient" les poids et hyperparamètres des meilleurs tout en les mutant légèrement. C'est une combinaison d'évolution génétique et de transfert de poids. Efficace pour des espaces de recherche très grands, mais nécessite des ressources de calcul importantes.

## Auto-ML : quand on délègue tout

Pour les cas où vous ne voulez pas du tout choisir manuellement l'architecture : [AutoKeras](#) explore l'espace d'architectures entier de façon automatique (couches, connexions, hyperparamètres). En production cloud, [Google Vertex AI Vizier](#) offre un service managé d'optimisation bayésienne à grande échelle. Ces outils font sens quand le problème est bien défini et que vous pouvez vous offrir des heures de recherche : en entreprise, ce n'est pas un luxe si le modèle final tourne pendant 2 ans.

---

On vient de voir comment chercher les bonnes valeurs d'hyperparamètres. Mais même avec la meilleure config trouvée, un modèle peut encore overfitter : il mémorise le train set au lieu d'en extraire la structure. Le tuning améliore la capacité du modèle, la régularisation la contrôle. Et ce contrôle, c'est ce qui sépare un modèle qui performe en validation d'un modèle qui tient en production. C'est l'objet de la section suivante.

---

## 5 - Régularisation : L1, L2, Batch Normalization, Early Stopping

Scénario : vous lancez l'entraînement, vous attendez 200 epochs, vous regardez les courbes.

```
train_loss : 0.05    val_loss : 0.38
train_acc  : 0.98    val_acc  : 0.74
```

L'écart est là. Le modèle a mémorisé le train set au lieu d'en extraire les patterns qui généralisent. C'est de l'overfitting, et maintenant la vraie question : vous changez l'architecture ou vous régularisez d'abord ?

La réponse quasi universelle : régulariser d'abord. Changer l'architecture prend du temps et peut introduire de nouveaux problèmes. La régularisation, c'est une ligne de code. Et dans 80% des cas, ça suffit à ramener `val_loss` et `train_loss` dans un écart raisonnable.

Encore faut-il savoir avec quoi. On a quatre outils majeurs ce matin : L1, L2, Batch Normalization, Early Stopping. Et Dropout, qui appartient à cette famille même si on l'a sous-estimé jusqu'ici.

## L'essentiel

### Régularisation L1 : pousser vers la sparsité

La **régularisation L1** ajoute la somme des valeurs absolues des poids à la loss totale. Le modèle minimise deux choses en même temps : l'erreur sur les données, et la taille absolue de ses poids.

Formule : `loss_totale = loss_tâche + lambda * sum(|w|)`

**Ce que ça fait au gradient** : lors de la backpropagation, L1 ajoute `+lambda * sign(w)` au gradient de chaque poids. `sign(w)` vaut `+1` si le poids est positif, `-1` s'il est négatif. Résultat : chaque poids est poussé vers zéro d'une valeur constante à chaque étape. Contrairement à L2, la force de rappel ne dépend pas de la magnitude du poids, elle est uniforme. Conséquence directe : les petits poids, qui contribuent peu, finissent à exactement zéro. Le réseau devient sparse.

**Géométrie** : la pénalité L1 dessine un losange dans l'espace des poids (norme 1). La pénalité L2 dessine une sphère (norme 2). Quand l'algorithme cherche le minimum de la loss totale, il cherche le point de contact entre la surface de la loss-tâche et la pénalité. Avec le losange, ce contact tombe souvent sur un coin, et les coins correspondent à des coordonnées nulles : des poids à zéro.

Pour la démonstration mathématique avec les courbes de niveau, voir [Wikipedia : Regularization \(mathematics\)](https://fr.wikipedia.org/wiki/R%C3%A9gularisation_(math%C3%A9matiques)).

**Effet pratique, feature selection implicite** : après entraînement avec L1, un certain nombre de poids de la première couche tombent à zéro ou quasi-zéro. Ce sont les features que le modèle a décidé d'ignorer. On peut le vérifier directement :

```
# Après model.fit() avec L1, extraire les poids de la première couche
first_layer_weights = model.layers[0].get_weights()[0] # shape (n_features, n_neurons)

# Compter les poids quasi-nuls (en valeur absolue < seuil)
# Seuil à 1e-4 : poids "effectivement nuls" pour L1
n_killed = np.sum(np.abs(first_layer_weights) < 1e-4)
print(f"Features 'tuées' par L1 : {n_killed} / {first_layer_weights.shape[0]}")
```

Avec `lambda=0.01` sur Pima (8 features), vous verrez souvent 2-3 features réduites à quasi-zéro. Avec `lambda=0.1`, parfois 5-6. C'est le modèle qui vous dit "ces colonnes n'apportent rien".

**Quand utiliser L1** : quand vous avez beaucoup de features (50+) et suspectez que la plupart sont du bruit. Sur de petits datasets structurés comme Pima (8 features), L1 est rarement le premier choix.



```
from tensorflow.keras import regularizers

model = keras.Sequential([
    layers.Dense(64, activation='relu',
                 kernel_regularizer=regularizers.l1(0.01), # lambda = 0.01
                 input_shape=(8,)),
    layers.Dense(1, activation='sigmoid')
])
```

## Régularisation L2 : poids petits, jamais nuls

La **régularisation L2** (aussi appelée "weight decay") ajoute la somme des carrés des poids à la loss.

Formule :  $\text{loss\_totale} = \text{loss\_t\^ache} + \lambda \times \sum(w^2)$

**Ce que ça fait au gradient** : L2 ajoute  $+2 \times \lambda \times w$  au gradient. La force de rappel est proportionnelle à la valeur du poids : les gros poids sont fortement pénalisés, les petits poids peu. Résultat : les poids tendent vers zéro mais n'y arrivent jamais exactement (sauf si  $\lambda$  est énorme). Le modèle distribue l'importance sur toutes les features plutôt que de se concentrer sur quelques-unes.

**L'équivalence SGD** : avec un optimiseur SGD, L2-via-loss est mathématiquement équivalent à appliquer une décroissance directe sur les poids à chaque étape. La mise à jour devient :

$$w_{(t+1)} = (1 - 2 \times \lambda \times \text{lr}) \times w_t - \text{lr} \times \text{grad}(\text{loss\_t\^ache})$$

À chaque step, les poids sont multipliés par un facteur légèrement inférieur à 1 avant d'être mis à jour par le gradient. C'est pour ça qu'on appelle ça "weight decay" : les poids décroissent passivement à chaque étape.

**AdamW, c'est quoi ?** Avec Adam, cette équivalence disparaît. Adam divise le gradient par son historique d'amplitude (les moments), ce qui modifie l'effet de la pénalité L2. [Loshchilov & Hutter \(2019\)](#) ont montré le problème et introduit AdamW (`keras.optimizers.AdamW`), qui découple la régularisation de l'optimisation des gradients. En pratique sur J2 : L2 standard suffit. Mais si vous utilisez Adam pour fine-tuner un grand modèle, AdamW est la bonne pratique.

```
model = keras.Sequential([
    layers.Dense(64, activation='relu',
                 kernel_regularizer=regularizers.l2(0.01),
                 input_shape=(8,)),
    layers.Dense(1, activation='sigmoid')
])
```

## L1 vs L2, et quand les combiner

### Instant Cheat Sheet : régularisations

|                 | L1                               | L2                           |
|-----------------|----------------------------------|------------------------------|
| Pénalité        | $\lambda \times \sum( w )$       | $\lambda \times \sum(w^2)$   |
| Gradient ajouté | $+\lambda \times \text{sign}(w)$ | $+2 \times \lambda \times w$ |

|                     | L1                                      | L2                                   |
|---------------------|-----------------------------------------|--------------------------------------|
| Effet sur les poids | Certains tombent à zéro (sparse)        | Tous petits, aucun à zéro            |
| Géométrie           | Losange                                 | Sphère                               |
| Quand l'utiliser    | Beaucoup de features, feature selection | Cas général, distribuer l'importance |
| Code Keras          | <code>regularizers.l1(0.01)</code>      | <code>regularizers.l2(0.01)</code>   |

**Elastic Net, le meilleur des deux** : si vous voulez à la fois réduire les poids (L2) et éliminer les features inutiles (L1), combinez-les avec `l1_l2`. Une ligne :

```
# L1 + L2 simultanément : shrinkage ET feature selection
kernel_regularizer=regularizers.l1_l2(l1=0.01, l2=0.01)
```

**Elastic Net ?** Technique combinant L1 et L2. Le L2 stabilise l'entraînement (sans lui, L1 seul peut être instable quand plusieurs features corrélées se "disputent" la sparsité). Le L1 produit quand même de la sparsité. Cas d'usage typique : données génomiques ou financières avec beaucoup de features corrélées.

**Lambda trop élevé** : avec `l2(10.0)`, la pénalité écrase la loss tâche. Le modèle sous-apprend massivement pour garder ses poids proches de zéro. Commencer à `l2(0.001)`, observer les courbes, ajuster. Règle empirique : si `train_loss` et `val_loss` sont proches mais les deux élevées, lambda est trop fort. Si `train_loss` descend et `val_loss` remonte, lambda est insuffisant.

## Batch Normalization : stabiliser les activations

*Pourquoi un réseau profond converge si mal sans normalisation intermédiaire ?*

Voilà le problème : imaginez que la couche 1 produit des activations entre 0.001 et 0.01 au début de l'entraînement. La couche 2 reçoit ces petites valeurs et calibre ses poids en conséquence. Puis l'entraînement continue, les poids de la couche 1 changent, ses activations passent à 5-50. La couche 2 reçoit maintenant des valeurs 5000 fois plus grandes que pendant son calibrage. Elle doit se recalibrer. Pendant ce temps, la couche 3 fait de même. Ce phénomène, chaque couche qui voit sa distribution d'entrée changer à chaque batch, s'appelle le **covariate shift interne** ("internal covariate shift", terme de [Ioffe & Szegedy, 2015](#) qui ont introduit BatchNorm).

La **Batch Normalization** coupe ce problème à la racine : elle normalise les activations à la sortie de chaque couche, batch par batch.

Pour chaque feature dans un batch :  $x_{\text{norm}} = (x - \text{mean\_batch}) / \sqrt{\text{var\_batch} + \text{epsilon}}$ . Puis deux paramètres appris, `gamma` (scale) et `beta` (shift), permettent au réseau de défaire cette normalisation si c'est utile :  $y = \text{gamma} * x_{\text{norm}} + \text{beta}$ . En pratique le réseau apprend à garder une distribution raisonnable, pas forcément centrée-réduite.

**Internals** : BatchNorm maintient deux statistiques par feature, mises à jour à chaque batch via une moyenne exponentielle mobile :

```
running_mean = momentum * running_mean + (1 - momentum) * batch_mean
running_var = momentum * running_var + (1 - momentum) * batch_var
```

Pendant l'entraînement (`training=True`), on utilise les stats du batch courant. Pendant l'inférence (`training=False`), on utilise les `running_mean` et `running_var` accumulés. Pourquoi cette distinction ? En production, on prédit souvent sample par sample : un batch de taille 1 n'a pas de "variance de batch" significative. On utilise donc les stats globales apprises sur l'ensemble du train set.

### Effets pratiques :

- Entraînement plus stable : on peut pousser le learning rate plus haut.
- Convergence plus rapide : souvent 2-5x moins d'epochs pour atteindre le même résultat.
- Léger effet régularisant (moins besoin de Dropout dans certains cas).

```
model = keras.Sequential([
    layers.Dense(64, input_shape=(8,)),
    layers.BatchNormalization(),      # normaliser avant l'activation : ordre original
    layers.Activation('relu'),
    layers.Dense(32),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dense(1, activation='sigmoid')
])
```

**Débat sur le placement :** l'ordre original du papier 2015 : Dense -> BN -> Activation. [He et al. \(2016\)](#) ("Identity Mappings in Deep Residual Networks") ont argumenté pour l'ordre inverse : Dense -> Activation -> BN. L'argument : BN après activation normalise des distributions déjà écrêtées par ReLU (toutes positives), ce qui peut être sous-optimal. En pratique : les deux ordres donnent des résultats comparables sur données structurées. En Phase 8, vous testez les deux sur Wine Quality et vous mesurez.

**Taille de batch et stabilité :** BatchNorm estime la variance sur le batch courant. Avec `batch_size < 8`, l'estimation de la variance est trop bruitée pour être utile. En dessous de `batch_size=2`, vous risquez des NaN (variance nulle si tous les samples du batch sont identiques). Règle : BN exige `batch_size >= 16` pour être fiable.

### Alternatives à BatchNorm :

**LayerNorm** : normalise sur les features d'un seul sample (pas sur le batch). Standard dans les Transformers (BERT, GPT). Fonctionne avec `batch_size=1`.

**GroupNorm** : normalise sur des groupes de channels. Bon choix pour les petits batches en vision (detection, segmentation).

**InstanceNorm** : normalise chaque sample indépendamment, channel par channel. Utilisé en style transfer.

Pour J2 sur données structurées avec `batch_size=32` : BatchNorm. Les autres variantes reviennent en J3 (CNN) et J4 (RNN/Transformers).

**Piège fine-tuning (teaser J3) :** quand on reprend un modèle pré-entraîné pour le spécialiser sur un nouveau dataset (transfer learning, J3), les couches BatchNorm ont des `running_mean` et `running_var` calibrés sur le dataset original (ImageNet, par exemple). Si vous relancez l'entraînement avec un petit dataset et un petit batch, ces statistiques vont se corrompre rapidement. La bonne pratique : `layer.trainable = False` sur les couches BN lors du fine-tuning initial, ou utiliser `training=False` explicitement.

## Early Stopping : arrêter au bon moment

L'entraînement trop long est une forme d'overfitting silencieux. Les poids continuent de se spécialiser sur le train set après que val\_loss ait atteint son minimum. Vous voulez les poids du moment où val\_loss était la plus basse, pas les poids de la dernière epoch.

L'**Early Stopping** surveille une métrique de validation epoch par epoch et arrête l'entraînement quand elle cesse de s'améliorer. Avec `restore_best_weights=True`, le modèle revient automatiquement aux poids du meilleur epoch.

```
early_stopping = keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=20,                # 20 epochs sans amélioration avant d'arrêter
    restore_best_weights=True   # toujours : sans ça, on récupère les poids du DERNIER
                                epoch
)

history = model.fit(
    X_train, y_train,
    epochs=500,                # plafond large, early stopping coupe avant
    validation_split=0.2,
    callbacks=[early_stopping]
)

print(f"Arrêt à l'epoch {len(history.history['val_loss'])}")
print(f"Meilleur val_loss : {min(history.history['val_loss']):.4f}")
```

*"Mais j'ai pas compris, c'est quoi la différence entre early stopping et juste mettre moins d'epochs ?"*

Mettre moins d'epochs, c'est fixer un plafond arbitraire à l'aveugle. Peut-être que sur ce dataset avec cette architecture, le minimum de val\_loss est à l'epoch 47. Peut-être à l'epoch 283. Vous ne pouvez pas le savoir à l'avance. Early Stopping s'arrête en regardant les données réelles, pas une intuition. Et il restaure les poids du moment exact où val\_loss était au plus bas.

**Monitor : choisir la bonne métrique.** `val_loss` est le défaut, et c'est le bon choix dans la majorité des cas.

Quelques exceptions :

- `val_accuracy` : uniquement si vos classes sont équilibrées ET que l'accuracy est vraiment votre objectif métier.
- `val_AUC` (via `keras.metrics.AUC`) : pour les datasets déséquilibrés, l'AUC reflète mieux la capacité du modèle à distinguer les classes.
- Métriques custom F1 : si le coût d'un faux négatif est très différent du coût d'un faux positif (détection de fraude, diagnostic médical), définissez une métrique F1 et surveillez-la.

**Patience : la règle des 10-20%.** Avec `max_epochs=500`, une `patience=10` stoppe au premier plateau temporaire, même si le modèle allait continuer à descendre. La val\_loss oscille naturellement, surtout sur les premiers epochs. Une règle empirique solide : `patience ≈ 10-20% de max_epochs`. Avec 500 epochs max, `patience=50` est raisonnable. Avec 100 epochs max, `patience=15-20`.

`restore_best_weights=True` : sans cette option, vous récupérez les poids de la dernière epoch (ceux après que val\_loss ait remonté). C'est exactement les poids que vous ne voulez pas. Toujours activer cette option.

## Dropout : régularisation par bruit structurel

Le [Dropout](#) (Srivastava et al., 2014) est une technique de régularisation qui désactive aléatoirement une fraction  $p$  des activations à chaque forward pass pendant l'entraînement.

**Mécanisme :** à chaque batch, un masque binaire aléatoire est tiré. Les neurones "tombés" (valeur mise à zéro) ne participent pas au calcul de ce batch, ni à la mise à jour des gradients. Le masque change à chaque batch.

**Inverted Dropout :** Keras utilise par défaut le "inverted dropout". Les activations conservées sont rescalées par  $1/(1-p)$  pour que la valeur attendue reste la même. Pourquoi c'est important : pendant l'inférence, Dropout est désactivé et aucun rescaling n'est nécessaire. Sans inverted dropout, vous devriez rescaler toutes les activations manuellement à l'inférence.

**Training vs inférence :** Dropout est actif uniquement pendant l'entraînement. Keras gère ça automatiquement via le flag `training`. Si vous appelez `model.predict()` ou `model(X, training=False)`, Dropout est désactivé, toutes les activations passent.

### Valeurs typiques :

- Couches cachées (Dense) :  $p = 0.2$  à  $0.5$ . Commencer à 0.3.
- Ne jamais mettre sur la couche de sortie.
- Avec Batch Normalization : l'usage combiné est débattu. BN + Dropout peut donner des résultats instables sur certaines architectures. En pratique : tester les deux séparément avant de les combiner.

**Position dans l'architecture :** après l'activation, avant la couche suivante.

```
model = keras.Sequential([
    layers.Dense(128, input_shape=(8,)),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.3),           # désactive 30% des neurones à chaque batch en train
    layers.Dense(64),
    layers.Activation('relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])
```

**SpatialDropout et AlphaDropout :** deux variantes pour des cas spécifiques.

**SpatialDropout :** pour les CNN. Au lieu de désactiver des neurones individuels, désactive des channels entiers. Préserve la cohérence spatiale.

**AlphaDropout :** conçu pour les réseaux SELU. SELU a des propriétés d'auto-normalisation que le Dropout standard détruit. AlphaDropout préserve mean et variance des activations.

## Combiner les régularisations

Toutes les régularisations ne sont pas compatibles de la même façon. Ce tableau indique ce qui fonctionne bien ensemble, ce qui est redondant, et ce qu'on évite :

|                | L2                | BatchNorm                              | Early Stopping                   | Dropout                     |
|----------------|-------------------|----------------------------------------|----------------------------------|-----------------------------|
| L2             | -                 | Souvent redondant (BN régularise déjà) | Duo de base, fonctionne toujours | Compatible, bon par défaut  |
| BatchNorm      | Souvent redondant | -                                      | Toujours compatible              | Instable, tester séparément |
| Early Stopping | Duo de base       | Toujours compatible                    | -                                | Toujours compatible         |
| Dropout        | Compatible        | Instable, tester                       | Toujours compatible              | -                           |

**Combinaison de départ recommandée** : L2 + Early Stopping. Fonctionne dans 80% des cas sans réglage fin.

**Ajouter BatchNorm quand** : réseau profond (4+ couches), convergence instable, learning rate idéal qui semble très faible.

**Ajouter Dropout quand** : overfitting persistant malgré L2, ou réseau très large par rapport à la taille du dataset.

**L1 quand** : beaucoup de features (50+) et suspicion de bruit dominant. Rare sur les datasets de J2.

## Creusons

### Dropout = approximation bayésienne

En 2016, [Gal & Ghahramani](#) ont montré que l'utilisation de Dropout à l'inférence (en gardant le masque actif sur plusieurs forward passes) est mathématiquement équivalente à une approximation de l'inférence bayésienne. Conséquence : en faisant N prédictions avec Dropout actif et en moyennant les résultats, on obtient une estimation de l'incertitude du modèle.

C'est le **MC Dropout** (Monte Carlo Dropout). En pratique : on appelle `model(X, training=True)` 50 fois sur le même input, et on regarde la variance des prédictions. Une variance faible = le modèle est confiant. Une variance élevée = le modèle hésite. C'est une technique d'uncertainty quantification sans avoir à entraîner un modèle bayésien complet. On y reviendra si J5 aborde la mise en production des modèles.

### DropConnect et Stochastic Depth

**DropConnect** (Wan et al., 2013) : au lieu de désactiver des activations, désactive des connexions (poids) aléatoirement. Effet similaire à Dropout mais encore plus agressif. Rarement utilisé en production.

**Stochastic Depth** (Huang et al., 2016) : désactive des couches entières aléatoirement pendant l'entraînement. Pensez-y comme du Dropout appliqué à des blocs résiduels. Utilisé dans les EfficientNets modernes.

### Data augmentation comme régularisation implicite

La data augmentation - créer artificiellement de nouveaux exemples à partir des exemples existants (rotations, flips, croppings pour les images) - est une forme de régularisation : elle empêche le modèle de mémoriser des patterns trop spécifiques au train set en lui montrant des variantes. Sur données structurées, l'équivalent est le bruit gaussien (`layers.GaussianNoise(stddev=0.1)`), l'ajout de petites perturbations aléatoires aux features pendant l'entraînement.

On revient sur la data augmentation image en J3 dans le contexte des CNN, où elle est presque obligatoire dès qu'on travaille avec peu de données.

---

On a maintenant tous les outils pour passer de "ça overfitte" à "ça généralise". L2 et Early Stopping pour commencer, BatchNorm si la convergence est chaotique, Dropout si l'overfitting résiste. Cet après-midi, vous allez les utiliser sur trois datasets avec des profils d'overfitting différents : California Housing (régression, overfit discret), Pima Diabetes (classification binaire, overfit fort), Wine Quality (multiclasse, problème de domination de classe). Phase 5 et Phase 8 sont les moments où ces régularisations entrent en scène. Vous ne choisissez pas une config au hasard : vous la choisissez en lisant les courbes TensorBoard que vous avez apprises à décoder en section 3.

---

## À votre tour

---

### Le projet

L'objectif de l'après-midi : construire trois classifieurs sur trois datasets réels, du chargement des données au modèle régularisé avec TensorBoard. Chaque dataset introduit un nouveau type de problème (régression, classification binaire, multiclassification), et chaque série de phases introduit une nouvelle technique (diagnostic TensorBoard, régularisation L2 + early stopping, keras-tuner, BatchNorm).

À la fin de l'après-midi, votre repo contient :

- Trois scripts Python indépendants, un par dataset
- Un dossier `logs/` avec les runs TensorBoard des trois entraînements
- Un tableau comparatif rempli de vos propres métriques (Phase 9)
- Un dataset de votre choix auquel vous avez appliqué le pipeline complet

Ce n'est pas un projet de démo. C'est un pipeline production-ready que vous pouvez mettre sur votre GitHub dès ce soir.

#### Rappel notation : votre repo, votre note

Votre repo GitHub est la source de vérité pour la notation. Ce que j'évalue :

- **Commits fréquents** : après chaque phase terminée, pas à la fin de l'après-midi. Un commit après la phase 1, un autre après la phase 2, etc.
- **Commits atomiques** : un commit = un changement logique. "Ajout du pipeline California Housing" est un bon message. "Modif" ne l'est pas. (et idéalement en anglais)
- **Staging explicite** : `git add pima_pipeline.py`, pas `git add .`. Inclure les fichiers de logs si vous les committez, pas les dossiers de tuning (plusieurs Mo).
- **Branches si variations** : si vous testez deux architectures différentes pour le même dataset, faites une branche.

---

## Phase 1 : Environnement et pipeline de données (California Housing)

Avant de coder : committez après cette phase (ex de message: `feat: pipeline chargement et normalisation California Housing`).

**Objectif** : charger le dataset [California Housing](#) (prédiction du prix médian des maisons en Californie), normaliser les features, faire un split train/val/test propre et vérifier les shapes.

California Housing : 20 640 exemples, 8 features numériques (revenu médian, âge des maisons, nombre de pièces, etc.), une cible continue (prix médian en centaines de milliers de dollars). Premier contact avec la régression : pas de classes, pas de softmax, juste un nombre à prédire.

```
Pipeline California Housing: load → split → scale → train → evaluate
```

**Décision en amont :** deux ordres possibles pour le pipeline. (a) `scaler.fit(X)` puis `split`. (b) `split` puis `scaler.fit(X_train)`. Les deux donnent un code qui tourne, mais un seul est correct en machine learning. Écrivez en tête de fichier, en commentaire, lequel vous choisissez et pourquoi. Indice si vous hésitez après plusieurs essais : comparez `X_test_norm.mean(axis=0)` entre les deux versions, l'écart vous donne la réponse.

Créez `phase1_pipeline_california.py` :

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Charger le dataset
housing = fetch_california_housing()
X, y = housing.data, housing.target

# StandardScaler (https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html)
# normalise chaque feature : (X - mean) / std.
# Résultat : mean = 0, std = 1 sur le train set.
# Pourquoi fitter sur X_train uniquement : si on fitte sur X entier,
# les stats du scaler "voient" le test set avant l'évaluation (data leakage).

# TODO : faire un premier split train/test avec test_size=0.2 et random_state=42
# TODO : faire un second split train/val sur le résultat précédent (val_size=0.2 du train)
# TODO : instancier un StandardScaler et le fitter sur X_train UNIQUEMENT
# TODO : transformer X_train, X_val, X_test avec le scaler fitté
# TODO : afficher les shapes de X_train, X_val, X_test
# TODO : afficher les stats descriptives de X_train_norm (mean et std par feature)
# TODO : afficher les feature_names du dataset ET vérifier qu'il y en a bien 8
```

Devrait afficher :

```
X_train shape : (13209, 8)
X_val shape    : (3303, 8)
X_test shape   : (4128, 8)
X_train_norm mean (par feature) : proche de 0.0 pour toutes les colonnes
X_train_norm std  (par feature) : proche de 1.0 pour toutes les colonnes
```

Tests à passer :

- Happy path : shapes correctes, stats descriptives montrent une normalisation propre (mean ~0, std ~1 sur le train). Si les 8 feature\_names s'affichent toutes, votre chargement est intact.



- Edge case : si vous fittez le scaler sur `x` entier avant le split, vous avez du data leakage. Le signe : `X_test_norm.mean()` sera proche de 0 comme le train, alors qu'il devrait en être légèrement éloigné. Testez les deux versions et comparez `X_test_norm.mean(axis=0)` dans chaque cas.
- Adversarial : ajoutez une ligne de données avec des valeurs aberrantes (`X_extreme = np.array([[99999, -99999, 0, 0, 0, 0, 37.0, -120.0]])`) et transformez-la avec votre scaler. Quelle valeur normalisée obtenez-vous pour `MedInc` ? Qu'est-ce que ça dit sur le comportement du modèle face aux outliers en production ?

## Phase 2 : Baseline PMC régression California Housing

Hop hop again -> les commits attention

**Objectif** : construire un PMC de régression, l'entraîner sur California Housing, et lire les métriques epoch par epoch. Première fois avec la compilation pour la régression : pas de `sigmoid`, pas de `binary_crossentropy`, et une loss qui s'appelle `mse`.

**MAE, c'est quoi ?** Mean Absolute Error : l'erreur moyenne en valeur absolue entre prédiction et cible. Ici, les cibles sont en centaines de milliers de dollars, donc MAE = 0.5 signifie 50 000 \$ d'erreur moyenne.

**À noter avant** `build_regression_model` : une variante proposée ajoute `activation='sigmoid'` sur la couche de sortie. Code valide Python, training qui tourne, loss qui descend. Et pourtant, modèle inutilisable. Pourquoi ? Posez la réponse en commentaire en tête de fonction, avant de coder.

Créez `phase2_baseline_regression.py` :

```
from tensorflow import keras
from tensorflow.keras import layers

def build_regression_model(input_dim):
    # TODO : instancier un modèle Sequential
    # TODO : ajouter une couche Dense(64, activation='relu', input_shape=(input_dim,))
    # TODO : ajouter une couche Dense(32, activation='relu')
    # TODO : ajouter la couche de sortie : Dense(1) sans activation (régression = valeur continue)
    # TODO : compiler avec optimizer='adam', loss='mse', metrics=['mae']
    # TODO : retourner le modèle
    pass

model = build_regression_model(input_dim=8)
model.summary()

# TODO : appeler model.fit avec X_train_norm, y_train, epochs=100, batch_size=32,
#         validation_data=(X_val_norm, y_val), verbose=1
#         stocker le retour dans `history`

# TODO : appeler model.evaluate sur (X_test_norm, y_test, verbose=0)
#         récupérer (test_loss, test_mae)
#         afficher avec f"MAE test : {test_mae:.4f} (en centaines de milliers de $)"
```

**Devrait afficher :** une `val_loss` qui descend sur les 50 premiers epochs, puis se stabilise. MAE test autour de 0.5-0.7 (soit 50 000 à 70 000 \$ d'erreur moyenne, ce qui est raisonnable pour un baseline sans tuning).

*Pourquoi MSE comme loss et MAE comme métrique ? MSE pénalise fort les grosses erreurs (carré), ce qui guide l'optimiseur à les éviter. MAE est plus lisible : 0.6 = 60 000 \$ d'erreur moyenne. Les deux coexistent.*

### Vérifications :

- Happy path : `val_loss` descend sur les premiers epochs et se stabilise, MAE test < 1.0.  
`model.summary()` affiche bien 3 couches, la dernière avec 1 neurone.
- Edge case : relancez avec `batch_size=1` (SGD pur) puis `batch_size=len(X_train_norm)` (Batch GD). Notez la différence de vitesse de convergence et de "nervosité" des courbes. SGD pur est très bruité, Batch GD est lisse mais lent. Laquelle des deux approches converge en moins d'epochs ?
- Adversarial : entraînez sans normalisation (passez `X_train` brut à la place de `X_train_norm`). Observez la MSE initiale : les features "Latitude" et "Longitude" (valeurs autour de 37-38) vs "AveRooms" (valeurs autour de 5-6) créent des gradients déséquilibrés. Jusqu'où monte la loss ? Ensuite, comparez les courbes Adam vs SGD avec `lr=0.001` sur les données normalisées : lequel converge en moins d'epochs ?

---

## Phase 3 : Diagnostic TensorBoard sur California Housing

Rappel : commits atomiques (ex: `feat: TensorBoard deux runs California Housing` une fois les deux runs lancés et visibles)

**Objectif :** brancher [TensorBoard](#) sur deux runs de California Housing (un avec normalisation, un sans) et comparer les scalars dans l'UI. TensorBoard est votre outil de diagnostic principal : à partir d'ici, plus de diagnostics à l'aveugle.

**Hypothèse à poser :** vous allez voir deux courbes (train\_loss, val\_loss). Trois situations classiques. (a) Les deux descendent ensemble, c'est sain. (b) train descend, val stagne ou remonte, c'est de l'overfitting. (c) val passe sous train, suspect, presque toujours un problème en amont (scaler appris sur tout, val trop facile, fuite). Notez en commentaire au bas du script dans laquelle de ces trois zones vous tombez, et votre interprétation. Indice si la situation (c) apparaît : revérifiez votre split.

Créez `phase3_tensorboard_california.py` :

```
import datetime
from tensorflow import keras

def train_with_tensorboard(X_train, y_train, X_val, y_val, run_name, epochs=100):
    """Entraîne un modèle de régression avec un callback TensorBoard horodaté."""
    # TODO : construire log_dir = f"logs/fit/{run_name}_" + timestamp au format HHMMSS
    #           (utiliser datetime.datetime.now().strftime("%H%M%S"))
    # TODO : instancier keras.callbacks.TensorBoard(log_dir=log_dir, histogram_freq=1)
    #           histogram_freq=1 : enregistrer les distributions des poids à chaque epoch
    # TODO : instancier le modèle via build_regression_model(input_dim=8)
    # TODO : appeler model.fit avec callbacks=[tb_callback],
    #           validation_data=(X_val, y_val), epochs=epochs, batch_size=32, verbose=0
    # TODO : afficher un message confirmant le run et son chemin de logs
    #           ex : f"Run '{run_name}' terminé. Logs dans {log_dir}"
    # TODO : retourner (model, history)
    pass
```

```
# Run 1 : données normalisées (bon comportement attendu)
model_norm, history_norm = train_with_tensorboard(
    X_train_norm, y_train, X_val_norm, y_val,
    run_name="california_norm"
)

# Run 2 : données brutes (comportement dégradé à observer)
model_raw, history_raw = train_with_tensorboard(
    X_train, y_train, X_val, y_val,
    run_name="california_raw"
)

# Lancer TensorBoard dans un terminal séparé :
# tensorboard --logdir=logs/fit
# Puis ouvrir http://localhost:6006
```

Dans TensorBoard, onglet Scalars : cochez les deux runs. Vous devez voir deux courbes avec des couleurs différentes. La `val_loss` de `california_raw` explose ou plafonne très haut, celle de `california_norm` descend proprement. C'est la même architecture, le même optimizer, la même durée : seule la normalisation change. C'est ça qui justifie le preprocessing.

*Pourquoi horodater les dossiers de logs ? Si vous relancez le même run sans timestamp, TensorBoard écrit dans le même dossier et mélange les courbes. L'horodatage garantit que chaque run a son propre espace.*

#### Ce qui doit marcher :

- Les deux runs apparaissent dans TensorBoard avec des noms distincts (`california_norm_HHMMSS` et `california_raw_HHMMSS`). Si vous ne voyez qu'un seul run, vérifiez que les deux appels à `train_with_tensorboard` ont bien été exécutés.
- Cas limite : lancez `tensorboard --logdir=logs/fit_vide` (dossier inexistant). TensorBoard démarre sans erreur bloquante mais affiche "No dashboards are active". C'est trompeur : on croit que les logs sont perdus alors qu'il suffit de corriger le chemin. Vérifiez toujours le chemin avant de conclure que l'entraînement a raté.
- Adversarial : tentez de lancer deux instances de TensorBoard en même temps sur le même port 6006. Que se passe-t-il ? L'une des deux refuse de démarrer (port déjà occupé). En production, ce genre de conflit silencieux bloque les pipelines de monitoring. Le réflexe : `pkill -f tensorboard` pour libérer le port avant de relancer.

## Phase 4 : Données et baseline (Pima Diabetes, classification binaire)

**Côté repo :** idée de commit: `feat: baseline PMC classification binaire Pima.`

Vous avez peut-être déjà touché Pima en J1 Phase 8. Aujourd'hui on ne refait pas le baseline pour le plaisir : on le pose proprement comme point de départ mesurable pour la régularisation (Phase 5) et le tuning automatique (Phase 6). Sans ce chiffre de référence, impossible de savoir si L2 améliore quoi que ce soit.

Le dataset [Pima Indians Diabetes](#) contient 768 femmes d'origine Pima avec 8 features médicales (glucose, IMC, âge, nombre de grossesses...) et une cible binaire : diabétique (1) ou non (0). Distribution : environ 65% non-diabétiques, 35% diabétiques. Ce déséquilibre n'est pas dramatique, mais il suffit à rendre une accuracy de 65% trompeuse si le modèle ne prédit que la classe majoritaire.

**Avant de célébrer l'accuracy :** Pima est déséquilibré. Calculez en tête de script, en commentaire, la répartition des deux classes (`np.bincount(y)` suffit). Puis posez la question avant même de lancer le training : un modèle qui prédirait toujours la classe majoritaire afficherait quelle accuracy ? Notez votre réponse. Vous comparerez avec votre vrai modèle ensuite.

Créez `phase4_pima_baseline.py` :

```
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Chargement Pima (URL directe, pas de compte Kaggle requis)
pima_url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
cols = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
        'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']
df = pd.read_csv(pima_url, names=cols)

# TODO : afficher df['Outcome'].value_counts() pour voir la distribution de classes
#         noter les proportions exactes avant d'entraîner quoi que ce soit

# TODO : afficher (df == 0).sum() pour toutes les colonnes
#         Glucose=0, BMI=0, Insulin=0, SkinThickness=0 sont physiologiquement impossibles
#         ce sont des NaN déguisés en zéros, encodage courant dans les datasets médicaux anciens
#         on les laisse pour l'instant, mais les noter : c'est un point de fragilité réel

X = df.drop('Outcome', axis=1).values
y = df['Outcome'].values

# TODO : split train/test 80/20, random_state=42
# TODO : instancier StandardScaler, le fitter sur X_train UNIQUEMENT, transformer train et test
#         (fitter sur X_test = data leakage : le modèle verrait les stats du test avant évaluation)

# TODO : construire un modèle Sequential binaire
#         architecture : Dense(64, relu, input_shape=(8,)) -> Dense(32, relu) -> Dense(1, sigmoid)
#         compiler avec optimizer='adam', loss='binary_crossentropy', metrics=['accuracy']

# TODO : entraîner 100 epochs, validation_split=0.2, batch_size=32
#         stocker le résultat dans une variable `history`
```

```
# TODO : afficher la val_accuracy finale (max sur toutes les epochs)
#         et vérifier que model.predict(X_val).mean() est proche de 0.35 (pas 0.05)
```

### Devrait afficher :

```
Distribution classes :
0      500   (65.1%)
1      268   (34.9%)

Colonnes avec des zéros suspects :
Glucose           5
BloodPressure     35
SkinThickness     227
Insulin           374
BMI               11
...

Baseline val_accuracy : ~0.73 à 0.77 (varie selon l'initialisation)
```

### Tests de validation :

- Happy path : val\_accuracy dépasse 70% sur le baseline non régularisé. Si c'est le cas, le modèle a appris quelque chose de réel, pas juste la fréquence de base des classes.
- Edge case : Pima encode les valeurs manquantes en 0 dans Glucose, BMI, Insulin et SkinThickness. Si vous remplacez ces zéros par la médiane de chaque colonne (`df[col].replace(0, df[col][df[col] != 0].median())`), la val\_accuracy change-t-elle ? De combien ? Garder la trace de la réponse.
- Adversarial : un modèle stupide qui prédit toujours 0 (non-diabétique) atteint 65% d'accuracy sans rien apprendre. Si votre baseline tombe à 65%, c'est un red flag. Vérifier : `model.predict(X_test).mean()` devrait être proche de 0.35 (le modèle voit des cas positifs). Si c'est proche de 0.05, le modèle est en train de préférer la classe majoritaire. Dans ce cas, relancer avec `class_weight={0: 1.0, 1: 1.9}` pour rééquilibrer.

## Phase 5 : Régularisation et tuning sur Pima

Commit (ex: `feat: L2 + early stopping Pima`)

Le baseline est posé. On l'a maintenant, ce chiffre de référence, et l'objectif de cette phase est simple : faire mieux, ou au moins comprendre pourquoi on n'y arrive pas. L2 + Early Stopping d'abord, Dropout ensuite. On compare les trois courbes côte à côte pour voir ce que chaque technique apporte réellement sur Pima.

Technique de régularisation introduite par [Srivastava et al. \(2014\)](#) : à chaque batch pendant l'entraînement, une fraction `p` des neurones est désactivée aléatoirement (mise à zéro). Vous avez le nom ?... Le **Dropout**. Le masque change à chaque batch, forçant le réseau à ne pas dépendre d'un seul neurone. Pendant l'inférence (`model.predict`), Dropout est automatiquement désactivé par Keras.

**Cible déclarée d'abord** : la phase 4 a posé le problème, ici on attaque. Écrivez en tête de fichier, en commentaire, votre objectif chiffré (par exemple, "recall classe 1 supérieur de 5 points au baseline"). Listez ensuite, toujours en commentaire, les leviers que vous allez essayer (L2, dropout, `class_weight`, seuil de décision) dans l'ordre que vous comptez suivre, et pourquoi cet ordre. Si vous battez votre cible, posez-en une

nouvelle (F1 macro, par exemple). Pas de plafond ici.

Créez `phase5_pima_regularisation.py` :

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers

# X_train_norm, X_val, y_train, y_val issus de la Phase 4
# (ou reconstruire le preprocessing en tête de fichier)

def build_pima_regularized(l2_lambda=0.01, use_dropout=False):
    """
    Modèle Pima avec régularisation L2 optionnelle et Dropout optionnel.
    Si use_dropout=True, insère un Dropout(0.3) après chaque couche cachée.
    """
    # TODO : créer un keras.Sequential vide
    # TODO : ajouter Dense(64, relu, input_shape=(8,)) avec
    kernel_regularizer=regularizers.l2(l2_lambda)
    # TODO : si use_dropout est True, ajouter layers.Dropout(0.3) immédiatement après
    # TODO : ajouter Dense(32, relu) avec kernel_regularizer=regularizers.l2(l2_lambda)
    # TODO : si use_dropout est True, ajouter layers.Dropout(0.3) immédiatement après
    # TODO : ajouter Dense(1, activation='sigmoid') en couche de sortie (pas de
    régularisation ici)
    # TODO : compiler avec optimizer='adam', loss='binary_crossentropy', metrics=
    ['accuracy']
    # TODO : retourner le modèle
    pass

# Callback Early Stopping : surveille val_loss, arrêt si pas d'amélioration pendant 15
epochs.
# restore_best_weights=True récupère les poids du meilleur epoch, pas du dernier.
early_stopping = keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=15, restore_best_weights=True
)

# --- Configuration 1 : baseline non régularisé (architecture identique à Phase 4) ---
# TODO : construire model_baseline avec build_pima_regularized(l2_lambda=0.0,
use_dropout=False)
# TODO : appeler model_baseline.fit(X_train_norm, y_train, epochs=300,
validation_split=0.2,
#                                     callbacks=[early_stopping], verbose=0)
# TODO : stocker le retour dans history_baseline
# TODO : afficher l'epoch d'arrêt (len(history_baseline.history['val_loss'])) et
max(val_accuracy)

# --- Configuration 2 : L2 seul (l2_lambda=0.01, use_dropout=False) ---
# TODO : construire model_l2 avec build_pima_regularized(l2_lambda=0.01,
use_dropout=False)
# TODO : entraîner avec les mêmes paramètres et le même callback
# TODO : stocker dans history_l2, afficher l'epoch d'arrêt et le max val_accuracy

# --- Configuration 3 : L2 + Dropout (l2_lambda=0.01, use_dropout=True) ---
```

```
# TODO : construire model_l2_drop avec build_pima_regularized(l2_lambda=0.01,
use_dropout=True)
# TODO : entraîner et stocker dans history_l2_drop
# TODO : afficher l'epoch d'arrêt et le max val_accuracy

# TODO : tracer les trois courbes val_loss côte à côte avec matplotlib
#         titres : "Baseline", "L2 seul", "L2 + Dropout"
#         ajouter une ligne verticale à l'epoch d'arrêt pour chaque config
#         sauvegarder en phase5_pima_3configs.png
```

Les courbes superposées vous diront les trois choses qu'on veut lire : est-ce que L2 réduit l'écart train/val ? Est-ce que Early Stopping s'active avant les 300 epochs ? Est-ce que Dropout apporte quelque chose de plus que L2 seul, ou est-ce redondant sur un dataset de cette taille ?

**Devrait afficher :** Early Stopping actif sur les deux configs régularisées, arrêt bien avant 300 epochs.

Val\_accuracy légèrement améliorée par rapport au baseline (entre 0.75 et 0.80 selon la config et l'initialisation).

### Vérifications :

- Happy path : Early Stopping s'active, epochs réels < 300, val\_loss stabilisée, val\_accuracy au moins aussi bonne que le baseline.
- Edge case : forcer `patience=1` et relancer. Le modèle s'arrête au premier plateau temporaire de val\_loss, probablement après 5-10 epochs seulement. Val\_accuracy plonge. C'est l'underfitting par arrêt prématuré. Comparer le nombre d'epochs réels avec `patience=1` vs `patience=15` : l'écart illustre pourquoi le réglage de patience n'est pas anodin.
- Adversarial : passer `l2_lambda=10.0` dans `build_pima_regularized`. La pénalité écrase la loss tâche. Val\_accuracy tombe vers 65%, le modèle prédit quasi toujours la classe majoritaire. C'est l'underfitting par excès de régularisation. Afficher `model_l2.layers[0].get_weights()[0].mean()` pour voir que les poids sont proches de zéro : la contrainte est si forte que le réseau n'a rien appris.

## Phase 6 : Recherche d'hyperparamètres sur Pima avec keras-tuner

Committing again (`feat: keras-tuner RandomSearch Pima`)

On a régularisé à la main en Phase 5. C'est instructif, mais en pratique, choisir `l2_lambda=0.01`, `units=64`, `activation='relu'`, `learning_rate=0.001` à la main, c'est parier sur une intuition. [keras-tuner](#) automatise cette recherche : on définit un espace d'hyperparamètres, et il explore cet espace de façon systématique sur un budget de trials fixé.

**RandomSearch, c'est quoi ?** Stratégie de recherche d'hyperparamètres qui échantillonne aléatoirement dans l'espace défini. Plus efficace que la grid search quand l'espace est grand : [Bergstra & Bengio \(2012\)](#) ont montré que random search trouve de meilleures configurations que grid search en un nombre d'essais identique, parce qu'une seule dimension peut dominer les performances (les autres sont presque indifférentes) et la grid search gaspille des essais sur les dimensions sans effet.

**Le piège du tuner :** un tuner mal paramétré ment avec assurance. Si vous laissez `max_epochs` trop bas, il sélectionne l'architecture qui converge vite, pas la meilleure. Notez en commentaire dans votre script comment vous comptez vous protéger : combien d'epochs minimum vous fixez pour valider un essai, et comment vous saurez que vous avez assez tourné. Comparez ensuite la "best architecture" trouvée avec un `max_epochs` court puis avec un `max_epochs` long. Surprise probable.

Créez `phase6_pima_kerastuner.py`:

```
import keras_tuner as kt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# X_train_norm, y_train issus de la Phase 4

def build_pima_model(hp):
    """
    HyperModel Pima : keras-tuner échantillonne les valeurs dans l'espace défini ici.
    hp.Int, hp.Float, hp.Choice sont les trois types d'hyperparamètres supportés.
    Chaque appel à cette fonction construit un modèle avec une configuration différente.
    """
    model = keras.Sequential()

    # TODO : définir le nombre de neurones de la couche 1
    #       hp.Int('units_1', min_value=32, max_value=128, step=32)
    #       -> keras-tuner explorera 32, 64, 96, 128

    # TODO : définir le nombre de neurones de la couche 2
    #       hp.Int('units_2', min_value=16, max_value=64, step=16)
    #       -> keras-tuner explorera 16, 32, 48, 64

    # TODO : définir la fonction d'activation à tester
    #       hp.Choice('activation', values=['relu', 'tanh'])

    # TODO : définir le taux de dropout
    #       hp.Float('dropout_rate', min_value=0.0, max_value=0.5, step=0.1)
    #       -> 0.0 = pas de dropout, 0.5 = dropout agressif

    # TODO : définir le learning rate
    #       hp.Choice('learning_rate', values=[1e-4, 5e-4, 1e-3, 5e-3, 1e-2])

    # TODO : ajouter Dense(units_1, activation=activation, input_shape=(8,))
    # TODO : si dropout_rate > 0, ajouter layers.Dropout(dropout_rate)
    # TODO : ajouter Dense(units_2, activation=activation)
    # TODO : si dropout_rate > 0, ajouter layers.Dropout(dropout_rate)
    # TODO : ajouter Dense(1, activation='sigmoid')
    # TODO : compiler avec Adam(learning_rate=learning_rate), 'binary_crossentropy',
['accuracy']
    # TODO : retourner le modèle
    pass

# Tuner RandomSearch : 15 trials, seed fixé pour reproductibilité
tuner = kt.RandomSearch(
    build_pima_model,
    objective='val_accuracy',
    max_trials=15,
    seed=42,
    directory='tuning_pima',
    project_name='pima_random'
```



```

)

# Résumé de l'espace de recherche (5 dimensions, 15 trials)
tuner.search_space_summary()

early_stop = keras.callbacks.EarlyStopping(monitor='val_loss', patience=10)

tuner.search(
    X_train_norm, y_train,
    epochs=100,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=0
)

# --- Analyse des résultats ---

# Récupérer les meilleurs hyperparamètres
best_hp = tuner.get_best_hyperparameters()[0]
print("Meilleur learning_rate :", best_hp.get('learning_rate'))
print("Meilleures units_1 :", best_hp.get('units_1'))
print("Meilleures units_2 :", best_hp.get('units_2'))
print("Meilleure activation :", best_hp.get('activation'))
print("Meilleur dropout_rate :", best_hp.get('dropout_rate'))

# TODO : appeler tuner.results_summary(num_trials=5) pour afficher les 5 meilleurs trials
#         avec leurs métriques respectives

# TODO : itérer sur tuner.get_best_hyperparameters(num_trials=5) et afficher,
#         pour chaque trial, les valeurs de chaque hyperparamètre
#         ex : for hp in tuner.get_best_hyperparameters(num_trials=5): print(hp.values)
#         chercher les invariants : y a-t-il des hyperparamètres qui reviennent toujours
#         dans les mêmes plages chez les 5 meilleurs ? C'est là que le signal est
#         concentré.

# Entraîner le meilleur modèle jusqu'à convergence complète
best_model = tuner.hypermodel.build(best_hp)
history_best = best_model.fit(
    X_train_norm, y_train,
    epochs=200,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=0
)

print(f"Best model val_accuracy : {max(history_best.history['val_accuracy']):.4f}")

```

**Devrait afficher :** un résumé de 15 trials avec les métriques de chacun, les hyperparamètres du meilleur trial, et une val\_accuracy finale comparable ou supérieure aux configs manuelles de Phase 5.

### Tests à passer :

- Happy path : 15 trials terminés sans erreur, `tuner.results_summary(5)` affiche les 5 meilleurs avec leurs configs respectives. Le meilleur trial dépasse val\_accuracy 0.76.

- Edge case : réduire à `max_trials=1`. Équivalent à prendre une config aléatoire sans chercher. Le code tourne, mais la "meilleure" config est la première essayée. Utile pour valider que le pipeline fonctionne rapidement, inutile pour le tuning réel. Mesurer la différence de `val_accuracy` entre `max_trials=1` et `max_trials=15` pour quantifier le gain de la recherche.
- Adversarial : remplacer `hp.Choice('learning_rate', values=[1e-4, 5e-4, 1e-3, 5e-3, 1e-2])` par `hp.Float('learning_rate', min_value=10.0, max_value=100.0)`. Le tuner explore une zone de learning rates catastrophiques : tous les modèles divergent ou oscillent. Observer que toutes les `val_accuracy` tombent vers 65% (prédiction de la classe majoritaire). keras-tuner ne plante pas : il choisit le "moins mauvais" parmi des configurations toutes mauvaises. C'est le signal que l'espace de recherche est mal défini, pas que le tuner est cassé.
- Stabilité : relancer le tuner avec `seed=43` au lieu de `seed=42`. Le best trial change-t-il ? De combien change la `val_accuracy` du meilleur modèle ? Si la variance entre deux seeds est supérieure à 2 points de %, le dataset est probablement trop petit pour que la recherche converge de façon fiable sur 15 trials. Tracer la distribution des `val_accuracy` des 15 trials pour les deux seeds côte à côte.

## Phase 7 : Données et baseline (Wine Quality, multiclassification)

Committez une fois la baseline stable, ex: `feat: baseline PMC multiclass Wine Quality`.

**Objectif :** charger le [Wine Quality UCI dataset](#) (vin rouge, 1 599 exemples, 11 features chimiques), agréger les 10 niveaux de qualité en 3 classes (low/medium/high), et entraîner un premier PMC multiclass avec softmax et [Sparse Categorical Crossentropy](#).

**Sparse Categorical Crossentropy ?** Version de la cross-entropie qui accepte les labels entiers directement (0, 1, 2...) sans passer par un one-hot encoding. Plus pratique que `categorical_crossentropy` quand on a des dizaines de classes.

Le dataset a un gros déséquilibre : la classe "medium" (qualité 5-6) représente plus de 80% des exemples. C'est exactement ce qu'on cherche à gérer proprement avec `stratify`.

**Choix à acter :** Wine Quality a une cible entre 3 et 8. Trois approches défendables. Classification multiclass (6 classes indépendantes). Régression (un nombre à prédire). Classification ordinaire (rare en Keras de base). Écrivez en tête de fichier, en commentaire, votre choix et ce que vous perdez avec chaque alternative. Construisez le modèle correspondant. Si vous bouclez vite, implémentez une seconde approche et comparez vos métriques sur le test set.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Chargement direct depuis UCI
wine_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"
df_wine = pd.read_csv(wine_url, sep=';')

print("Distribution des qualités brutes :")
```

```

print(df_wine['quality'].value_counts().sort_index())

# Agrégation en 3 classes
def map_quality(q):
    if q <= 4:
        return 0 # low
    elif q <= 6:
        return 1 # medium
    else:
        return 2 # high

df_wine['quality_3class'] = df_wine['quality'].apply(map_quality)

print("\nDistribution agrégée (3 classes) :")
print(df_wine['quality_3class'].value_counts().sort_index())

X_wine = df_wine.drop(['quality', 'quality_3class'], axis=1).values
y_wine = df_wine['quality_3class'].values

# TODO : split 80/20 avec stratify=y_wine
#         (sans stratify, le déséquilibre peut produire un train set sans aucun exemple
#         "low")
# TODO : normaliser avec StandardScaler fitté sur X_wine_train uniquement
# TODO : construire un PMC multiclass : Dense(64, relu) -> Dense(32, relu) -> Dense(3,
#         softmax)
# TODO : compiler avec sparse_categorical_crossentropy et metric=['accuracy']
# TODO : entraîner 100 epochs, validation_split=0.2, et afficher val_accuracy finale

```

### Devrait afficher :

```

Distribution agrégée : 0 (low) : ~63, 1 (medium) : ~1319, 2 (high) : ~217
val_accuracy baseline : ~0.60 à 0.70 (la classe medium domine, le modèle a tendance à la
sur-prédire)

```

**Ordinal vs nominal** : la qualité du vin est en réalité une variable ordinale ( $3 < 4 < 5 < \dots < 8$ ). En traitant les 3 classes comme des catégories indépendantes (softmax + sparse\_cce), on perd cette information d'ordre : "low" et "high" ont la même distance que "low" et "medium". Une approche plus rigoureuse serait l'[Ordinal Regression](#), qui encode explicitement l'ordre dans la structure du modèle. Hors programme pour nous, mais c'est un vrai débat en ML pour les datasets de rating, de scoring, et de satisfaction client.

### Vérifications :

Les trois scénarios à tester avant de passer à la suite :

- Scénario normal : val\_accuracy > 50% sur les 3 classes agrégées. Un classificateur naïf (toujours prédire "medium") plafonne à ~82% d'accuracy... mais ne détecte jamais les vins low ni high. Regardez la matrice de confusion : si votre modèle ne prédit jamais les classes 0 ou 2, c'est qu'il a lui aussi capitulé.
- Cas limite : essayez sans l'agrégation, directement sur les 6 qualités brutes (3 à 8). La val\_accuracy chute typiquement à 40-45%, et la classe 5 (majorité absolue) écrase tout. C'est la démonstration concrète de pourquoi l'agrégation de classes déséquilibrées aide pour établir une baseline solide.

- Scénario adversarial : utilisez `categorical_crossentropy` au lieu de `sparse_categorical_crossentropy` sans adapter les labels. Keras lève `InvalidArgumentError: logits and labels must have the same first dimension`. C'est précisément pourquoi `sparse_categorical_crossentropy` est le choix par défaut quand les labels sont des entiers : on évite l'étape `to_categorical` et ses erreurs de shape silencieuses.

## Phase 8 : Tuning et régularisation sur Wine + Batch Normalization

**Côté repo :** committez une fois les 4 configs comparées : `feat: BatchNorm L2 Wine Quality comparison`.

**Objectif :** ajouter [BatchNormalization](#) au modèle Wine Quality, comparer la vitesse de convergence sur TensorBoard pour 4 configurations, et mesurer l'impact du positionnement de BN (avant vs après l'activation).

**BatchNormalization ?** Technique introduite par [Ioffe & Szegedy \(2015\)](#) qui normalise les activations de chaque couche à chaque batch (moyenne 0, variance 1), puis leur applique des paramètres apprenables (gamma, beta). Résultat : l'entraînement converge plus vite et tolère des learning rates plus élevés.

**Ordre à justifier :** vous allez empiler `Dense`, `BatchNormalization`, `ReLU`, `Dropout`. L'ordre n'est pas anodin, plusieurs combinaisons existent dans les papiers, deux donnent des résultats nettement différents. Notez en commentaire avant `build_model` l'ordre que vous adoptez et un argument pour. Validez en comparant `val_loss` avec un ordre alternatif sur quelques epochs.

Voici la signature et le squelette de la fonction de construction. L'objectif est de tester deux stratégies de placement du BN : avant ou après l'activation.

```
import datetime
from tensorflow.keras import regularizers

def build_wine_model(use_batchnorm=False, bn_before_activation=True, extra_layer=False):
    """
    PMC multiclass Wine Quality.
    bn_before_activation=True: Dense linéaire -> BN -> ReLU (recommandé Ioffe&Szegedy
    2015).
    bn_before_activation=False: Dense+ReLU -> BN (testé pour comparer).
    extra_layer: ajoute une 3e couche cachée (16 units) pour tester l'impact de la
    profondeur.
    """
    # TODO : créer un keras.Sequential vide
    # TODO : si use_batchnorm et bn_before_activation
    #         -> Dense(64, input_shape=(11,)) PUIS BatchNormalization() PUIS
    Activation('relu')
    # TODO : sinon -> Dense(64, activation='relu', input_shape=(11,))
    #         puis si use_batchnorm: BatchNormalization() après l'activation
    # TODO : ajouter Dense(32, activation='relu',
    kernel_regularizer=regularizers.l2(0.01))
    # TODO : si extra_layer, ajouter Dense(16, activation='relu')
    # TODO : ajouter Dense(3, activation='softmax')
    # TODO : compiler optimizer='adam', loss='sparse_categorical_crossentropy', metrics=
    ['accuracy']
    # TODO : retourner le modèle
    pass
```

Voici le squelette du runner de comparaison. Vous câblez la boucle d'entraînement et l'agrégation des résultats. Chaque config est une lambda (factory function) : `build_fn()` construit un modèle frais à chaque itération, ce qui évite de réutiliser accidentellement un modèle déjà entraîné.

```
early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=15, restore_best_weights=True
)

# Les 4 configurations à comparer (signatures déjà choisies)
configs = {
    'sans_bn': lambda: build_wine_model(use_batchnorm=False),
    'bn_avant_activation': lambda: build_wine_model(use_batchnorm=True,
bn_before_activation=True),
    'bn_apres_activation': lambda: build_wine_model(use_batchnorm=True,
bn_before_activation=False),
    'bn_extra_couche': lambda: build_wine_model(use_batchnorm=True,
bn_before_activation=True, extra_layer=True),
}

results = {}
for name, build_fn in configs.items():
    # TODO : construire un log_dir horodaté:
    log_dir = f"logs/wine/{name}_{datetime.datetime.now().strftime('%H%M%S')}"
    # TODO : instancier le callback TensorBoard avec ce log_dir
    # TODO : appeler build_fn() pour obtenir un modèle frais (chaque config a son propre
    # modèle)
    # TODO : entraîner avec X_wine_train, y_wine_train, 200 epochs, validation_split=0.2,
    # callbacks=[early_stop, tb_callback], verbose=0
    # TODO : extraire max(history.history['val_accuracy']) et
    # len(history.history['val_loss'])
    # TODO : stocker dans results[name] = {'val_accuracy': ..., 'epochs': ...}
    # TODO : print(f"{name}: val_accuracy=..., stopped at epoch ...")
    pass
```

**Devrait afficher :** les configs avec BN stoppent généralement plus tôt (early stopping déclenché après moins d'epochs réels). La val\_accuracy finale varie selon les seeds, mais la convergence est visible sur TensorBoard.

```
sans_bn: val_accuracy=0.72xx, stopped at epoch 87
bn_avant_activation: val_accuracy=0.73xx, stopped at epoch 61
bn_apres_activation: val_accuracy=0.71xx, stopped at epoch 74
bn_extra_couche: val_accuracy=0.74xx, stopped at epoch 58
```

*Dans TensorBoard onglet Scalars : 4 courbes de val\_loss superposées. Repérez à quelle epoch chaque config commence à plafonner. C'est une observation que le tableau de résultats seul ne donne pas.*

### Checkpoint qualité :

- Happy path : dans TensorBoard, la val\_loss de la config `bn_avant_activation` descend plus vite dans les 30 premiers epochs que `sans_bn`. Le nombre d'epochs réels (early stopping) doit être inférieur. Pas de règle absolue sur la val\_accuracy finale : l'accélération de convergence est le résultat à observer.

- Cas limite : lancez `build_wine_model(use_batchnorm=True)` avec un `batch_size=1` dans le fit. [BatchNormalization](#) calcule moyenne et variance sur le batch courant : avec un seul exemple, la variance est nulle et l'estimation est instable. Keras peut émettre des warnings ou produire une `val_loss` qui diverge. En pratique, BN demande un `batch_size` d'au moins 8 à 16 pour être stable.
- Scénario adversarial : comparez `bn_avant_activation` et `bn_apres_activation` sur 5 seeds différentes (`tf.random.set_seed(i)` pour `i in range(5)`). La configuration "gagnante" change-t-elle selon la seed ? Si oui, quelle conclusion sur la fiabilité d'une comparaison a seed unique ?

## Phase 9 : Agrégation cross-dataset et pipeline personnel

Des commits pour changer (comme: `feat: tableau comparatif 3 datasets` puis `feat: pipeline [nom dataset] custom`).

C'est la phase de synthèse. On agrège, on réfléchit, et on va plus loin avec un dataset qui vous appartient.

**Hypothèse avant tableau :** avant de remplir la moindre cellule, posez en commentaire dans un `README.md` à la racine de votre repo trois hypothèses. (a) Quel dataset profite le plus de la régularisation et pourquoi. (b) Lequel le moins, et pourquoi. (c) Quel facteur (taille, déséquilibre, dimensionnalité) explique le plus l'écart. Vous remplissez ensuite le tableau et confrontez. Si une hypothèse tient, expliquez en deux lignes pourquoi. Sinon, expliquez en deux lignes ce que vous aviez sous-estimé. Pas de plafond : enrichissez le README jusqu'à ce qu'il soit publiable sur votre GitHub.

### Partie A : tableau comparatif cross-dataset

Remplissez ce tableau avec vos propres résultats des phases 1 à 8. Les valeurs viennent de vos runs TensorBoard et de vos logs keras-tuner : personne n'a les mêmes chiffres, c'est votre tableau.

| Dataset            | Tâche                  | val_accuracy / val_MAE | val_loss finale | Epochs réels | Meilleur lr | Meilleure taille couche | Régularisation      |
|--------------------|------------------------|------------------------|-----------------|--------------|-------------|-------------------------|---------------------|
| California Housing | Régression             |                        |                 |              | N/A         |                         | Aucune (baseline)   |
| Pima Diabetes      | Classification binaire |                        |                 |              |             |                         | L2 + Early Stopping |
| Wine Quality       | Multiclassification    |                        |                 |              | N/A         |                         | L2 + BatchNorm      |

Une fois le tableau rempli, prenez 5 minutes pour répondre aux questions suivantes. Pas de réponse unique : l'objectif est de formuler une hypothèse et de la défendre avec vos chiffres.

- *Quel hyperparamètre a eu le plus d'impact sur vos résultats : le learning rate, la taille des couches, ou la régularisation ? Est-ce cohérent avec ce qu'on a vu sur la random search ce matin ?*
- *Sur quel dataset l'écart entre train\_accuracy et val\_accuracy était-il le plus grand ? Quelle explication architecturale ou liée aux données pourrait l'expliquer ?*
- *Quel dataset a déclenché l'early stopping le plus tôt (en epochs réels) ? Pourquoi ce dataset "converge" plus vite que les autres : est-ce la taille du dataset, le signal dans les features, ou le niveau de bruit ?*

Notez vos réponses dans un commentaire en bas de votre script ou dans votre `README.md`. Ce sont exactement les questions qu'un data scientist senior pose en revue de code.

#### Avant de boucler :

- Tableau complet : 3 lignes remplies avec des chiffres réels, tirés de vos runs TensorBoard et logs keras-tuner. Aucun champ vide, aucun "TODO". Vous avez rédigé une réponse aux 3 questions de réflexion ci-dessus.

- Cas limite : si pour un dataset Early Stopping n'a jamais déclenché (epochs réels = max\_epochs), c'est anormal. Soit `patience` est trop grande, soit le modèle continue de s'améliorer pendant les 200 ou 300 epochs prévus (rare mais possible). Vérifier la courbe val\_loss dans TensorBoard : converge-t-elle vraiment ou oscille-t-elle encore à l'epoch finale ?
- Adversarial : comparez votre val\_accuracy à celle d'un classificateur naïf qui prédit toujours la classe majoritaire. Sur Pima : 65% sans rien apprendre. Sur Wine 3 classes : environ 82% (medium domine). Si votre modèle ne bat pas le naïf, votre tableau est trompeur. Qu'est-ce qu'il faudrait ajouter comme colonne dans le tableau pour ne pas se mentir ?

## Partie B : pipeline sur dataset personnel

Choisissez un dataset et appliquez le pipeline complet : chargement, normalisation, split, PMC Keras, diagnostic TensorBoard, régularisation si le modèle overfitte, keras-tuner si vous avancez vite.

Options de départ :

- `sklearn.datasets.load_iris()` : multiclass 3 classes, 150 exemples, 4 features. Petit et rapide, bon pour vérifier que votre pipeline tourne avant de passer à quelque chose de plus gros.
- `sklearn.datasets.load_breast_cancer()` : binaire, 569 exemples, 30 features. Même structure que Pima, mais plus de features, donc plus d'occasions d'overfitting.
- `sklearn.datasets.load_digits()` : multiclass 10 classes, 1 797 exemples, 64 features (images 8x8 aplaties). C'est un pont direct vers la session J3 sur les CNN : ce que vous obtenez avec un PMC ici, les convolutions le surpassent demain.
- Un CSV Kaggle sur votre domaine : santé, finance, sport, e-commerce. Si vous avez déjà un jeu de données sur lequel vous voulez travailler, c'est le moment.

```
# Structure à compléter pour votre dataset

# TODO : charger le dataset de votre choix
# TODO : afficher shape, distribution des classes (ou stats de la target pour la
régression)
# TODO : split 80/20 stratifié si classification
# TODO : normaliser (StandardScaler fitté sur train uniquement)
# TODO : construire un PMC Keras adapté à la tâche (binaire / multiclass / régression)
# TODO : brancher TensorBoard
# TODO : entraîner, diagnostiquer les courbes, ajuster si overfitting ou underfitting
# TODO : ajouter regularizers.l2 ou EarlyStopping si nécessaire
# TODO : afficher : test accuracy (ou MAE), epochs réels, val_loss finale
```

Il n'y a pas d'état terminé ici. Une fois que votre pipeline tourne et que TensorBoard est branché :

- Comparez Adam vs SGD avec les memes hyperparamètres sur votre dataset. Lequel converge plus vite ? Lequel atteint une meilleure val\_accuracy finale ?
- Testez l'impact de la normalisation : entraînez sans `StandardScaler` et mesurez la différence en epochs nécessaires pour converger. Sur certains datasets, l'impact est marginal. Sur d'autres, le modèle diverge complètement.
- Lancez keras-tuner sur votre dataset. Les hyperparamètres optimaux ressemblent-ils a ceux de Pima ? Ou sont-ils différents ?

- Si vous avez choisi `load_digits()` : comparez votre `val_accuracy` PMC avec la valeur que vous obtiendrez demain avec un CNN sur les memes données. Gardez le chiffre quelque part.
- Ajoutez une entrée extrême dans votre set de test : `X_extreme = np.array([[99999] * n_features])`. Qu'est-ce que votre modèle prédit, et avec quelle confiance ? Une prédiction très confiante sur des données hors distribution est un signal d'alarme en production.

## Partie C : post-mortem rapide

Avant de fermer votre laptop, ajoutez 3 lignes dans votre `README.md` :

1. Qu'est-ce qui vous a surpris aujourd'hui ?
2. Qu'est-ce que vous refereriez différemment si vous recommenciez la journée ?
3. Qu'est-ce que vous n'avez pas eu le temps d'explorer ?

C'est une habitude que les équipes data prennent en fin de sprint. Pas un devoir, pas noté : juste une trace pour vous. Dans 6 mois, ce README sera votre meilleur aide-mémoire sur ce que vous avez vraiment appris ce jour-la, pas ce que le cours vous a dit d'apprendre.

---

**Pensez dès maintenant à votre projet J5.** Si votre sujet de coeur implique des données tabulaires (santé, finance, RH, e-commerce), le pipeline que vous venez de construire est exactement celui que vous allez réutiliser. Vérifiez que votre dataset est disponible sur Kaggle ou HuggingFace, que l'entraînement tient en moins de 20 min sur Colab T4, et que le résultat est déployable en WebApp. L'architecture (PMC, CNN, RNN) découlera du type de données.

---

## Récapitulatif Jour 2

### Ce qu'on a appris

1. **De numpy à Keras** : même PMC, 10x moins de code, même puissance. On comprend maintenant ce que Keras fait pour nous parce qu'on l'a fait à la main hier.
2. **TensorBoard** : brancher, nommer les runs, lire les scalars, comparer des expériences côte à côte. C'est l'outil que les équipes ML utilisent en production.
3. **Diagnostic** : reconnaître overfitting et underfitting sur les courbes train/val, et nommer ce qu'on voit.
4. **Tuning** : grid search, random search, keras-tuner. Et pourquoi le random souvent gagne à budget égal.
5. **Régularisation** : L1 pour la sparsité, L2 pour les petits poids, BatchNorm pour la stabilité de l'entraînement, Early Stopping pour arrêter au bon moment.
6. **Trois datasets, un pipeline** : régression, binaire, multiclasse. Même recette, paramétrisations différentes.

### Points d'attention pour demain

- Demain on quitte les données structurées pour les images. La logique reste : un problème, un pipeline, un diagnostic.
- Les CNN introduisent une nouvelle brique que le PMC n'a pas : la convolution. On va construire ça conceptuellement avant de coder.



- Les datasets vont peser des centaines de Mo. Vérifier que votre accès Colab fonctionne et créer un compte Kaggle ce soir si ce n'est pas fait (le dataset Cats vs Dogs fait 800 Mo).
- **Demain, cible : 95% d'accuracy sur Cats vs Dogs** avec transfer learning. Le PMC ne pourrait pas faire ça.